

SIEMENS EDA

Catapult[®] Synthesis QuickStart Guide

Software Version 2025.2_1
June 2025

SIEMENS

This Documentation contains trade secrets or otherwise confidential information owned by Siemens Industry Software Inc. or its affiliates (collectively, "Siemens"), or its licensors. Access to and use of this Documentation is strictly limited as set forth in Customer's applicable agreement(s) with Siemens. This Documentation may not be copied, distributed, or otherwise disclosed by Customer without the express written permission of Siemens, and may not be used in any way not expressly authorized by Siemens.

This Documentation is for information and instruction purposes. Siemens reserves the right to make changes in specifications and other information contained in this Documentation without prior notice, and the reader should, in all cases, consult Siemens to determine whether any changes have been made.

No representation or other affirmation of fact contained in this Documentation shall be deemed to be a warranty or give rise to any liability of Siemens whatsoever.

If you have a signed license agreement with Siemens for the product with which this Documentation will be used, your use of this Documentation is subject to the scope of license and the software protection and security provisions of that agreement. If you do not have such a signed license agreement, your use is subject to the Siemens Universal Customer Agreement, which may be viewed at <https://www.sw.siemens.com/en-US/sw-terms/base/uca/>, as supplemented by the product specific terms which may be viewed at <https://www.sw.siemens.com/en-US/sw-terms/supplements/>.

SIEMENS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS DOCUMENTATION INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT OF INTELLECTUAL PROPERTY. SIEMENS SHALL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, CONSEQUENTIAL OR PUNITIVE DAMAGES, LOST DATA OR PROFITS, EVEN IF SUCH DAMAGES WERE FORESEEABLE, ARISING OUT OF OR RELATED TO THIS DOCUMENTATION OR THE INFORMATION CONTAINED IN IT, EVEN IF SIEMENS HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

TRADEMARKS: The trademarks, logos, and service marks (collectively, "Marks") used herein are the property of Siemens or other parties. No one is permitted to use these Marks without the prior written consent of Siemens or the owner of the Marks, as applicable. The use herein of third party Marks is not an attempt to indicate Siemens as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A list of Siemens' Marks may be viewed at: www.plm.automation.siemens.com/global/en/legal/trademarks.html. The registered trademark Linux® is used pursuant to a sublicense from LMI, the exclusive licensee of Linus Torvalds, owner of the mark on a world-wide basis.

About Siemens Digital Industries Software

Siemens Digital Industries Software is a global leader in the growing field of product lifecycle management (PLM), manufacturing operations management (MOM), and electronic design automation (EDA) software, hardware, and services. Siemens works with more than 100,000 customers, leading the digitalization of their planning and manufacturing processes. At Siemens Digital Industries Software, we blur the boundaries between industry domains by integrating the virtual and physical, hardware and software, design and manufacturing worlds. With the rapid pace of innovation, digitalization is no longer tomorrow's idea. We take what the future promises tomorrow and make it real for our customers today. Where today meets tomorrow. Our culture encourages creativity, welcomes fresh thinking and focuses on growth, so our people, our business, and our customers can achieve their full potential.

Support Center: support.sw.siemens.com

Send Feedback on Documentation: support.sw.siemens.com/doc_feedback_form

Table of Contents

Chapter 1: Coding with C++	2
1.1. Compiling Designs with HLS Include Files	2
1.1.1. Including Header Files from the Catapult Install Tree	2
1.1.2. Setting Compiler Optimization Levels	2
1.2. Using the Algorithmic C Data Types	3
1.2.1. <i>ac_int</i> : Integer Data Types	3
1.2.2. <i>ac_fixed</i> : Fixed-Point Data Types	4
1.2.3. <i>ac_float</i> : Floating-Point Data Types	5
1.2.4. <i>ac_complex</i> : Complex Data Types	5
1.2.5. Pretty-Printing Bit-Accurate Data Types	5
1.3. Creating a Design Module with Interfaces	5
1.3.1. Class-based Coding Style	6
1.3.2. Data Interfaces	7
1.3.3. Configuration Interfaces	11
1.3.4. Copying Arrays from an Interface or Internally	12
1.4. Coding for Area	13
1.4.1. Mutually Exclusive Conditions	13
1.4.2. Making Things Constant	13
1.4.3. Loops	14
1.5. Coding for Performance and Area	16
1.5.1. Intra-Process Parallelism (Loop Unrolling)	16
1.5.2. Unrolling a Loop to Make the Loop Iterator Constant	17
1.5.3. Achieving Maximum Throughput	19
1.5.4. Inter-Process Concurrency	20
1.5.5. Coding Memory Architecture	24
1.6. Coding for Power	28
1.7. Using HLS Math Libraries	31
Chapter 2: Coding with SystemC	32
2.1. Compiling Designs with SystemC and MatchLib Include Files	32
2.1.1. Including Header Files from the Catapult Install Tree	33
2.1.2. Including Header Files after Downloading from HLSLibs.org	33
2.1.3. Setting Compiler Optimization Levels	33
2.1.4. Using the Algorithmic C Data Types	34
2.1.5. <i>ac_int</i> : Integer Data Types	34
2.1.6. <i>ac_fixed</i> : Fixed-Point Data Types	35

Table of Contents

2.1.7. <i>ac_float: Floating-Point Data Types</i>	36
2.1.8. <i>ac_complex: Complex Data Types</i>	36
2.1.9. <i>Pretty-Printing Bit-Accurate Data Types</i>	36
2.2. Creating a Design Module with Interfaces.....	36
2.2.1. <i>Understanding How HLS Synthesizes a SystemC Design</i>	36
2.2.2. <i>Coding SystemC Modules</i>	37
2.2.3. <i>Sequential Processes</i>	38
2.2.4. <i>Combinational Processes</i>	39
2.2.5. <i>Data Interfaces and Processes</i>	40
2.2.6. <i>Configuration Interfaces</i>	49
2.2.7. <i>Handling User-defined Structs</i>	50
2.3. Coding for Area.....	51
2.3.1. <i>Mutually Exclusive Conditions</i>	51
2.3.2. <i>Making Things Constant</i>	52
2.3.3. <i>Loops</i>	53
2.4. Coding for Performance and Area.....	53
2.4.1. <i>Intra-Process Parallelism (Loop Unrolling)</i>	54
2.4.2. <i>Unrolling a Loop to Make the Loop Iterator Constant</i>	55
2.4.3. <i>Achieving Maximum Throughput</i>	56
2.4.4. <i>Inter-Process Concurrency</i>	57
2.4.5. <i>Coding Memory Architecture</i>	60
2.5. Non-Blocking IO.....	63
2.6. Coding for Power.....	63
2.7. Using HLS Math Libraries.....	66
Chapter 3: Third-Party Information	67

Chapter 1: Coding with C++

This quick start guide is meant to give designers a jump-start for writing C++ code for High-Level Synthesis. It is by no means comprehensive and more in-depth coverage of coding style and general HLS optimizations can be found in the HLS Blue Book as well as the Catapult User Reference Manual. This document does provide links to the Catapult user documentation where appropriate.

- Compiling Designs with HLS Include Files
- Using the Algorithmic C Data Types
- Creating a Design Module with Interfaces
- Coding for Area
- Coding for Performance and Area
- Coding for Power
- Using HLS Math Libraries

1.1. Compiling Designs with HLS Include Files

Designing in Untimed C++ for HLS will usually require including library header files for things such as bit-accurate data types, streaming interfaces, etc. This requires that the C++ compiler know where to find these header files. There are two possible locations where the header files can be included and depends on whether the user has the Catapult HLS tool installed on their network.

1.1.1. Including Header Files from the Catapult Install Tree

If Catapult HLS is installed on the user's network an environment variable should be set to point to the Mgc_home directory in the install tree

```
setenv MGC_HOME <Path to Catapult install>/Mgc_home
```

Then designs can be compiled by including the Catapult include directory:

```
g++ -I $MGC_HOME/shared/include <design file name>.cpp
```

1.1.2. Setting Compiler Optimization Levels

Compiler optimizations will affect the execution runtime of the users C++ program

Debugging C++ Designs

Compiler optimizations should be disabled when designing and debugging using the bit-accurate data types. This will allow designers to step through the code in a C++ debugger such as VSCode or Eclipse. Optimizations can be disabled by setting the optimization level to zero (-O0)

```
g++ -O0 -I $MGC_HOME/shared/include <design file name>.cpp
```

If the user wants the fastest execution time, they should set the optimization level to -O3

```
g++ -O3 -I $MGC_HOME/shared/include <design file name>.cpp
```

1.2. Using the Algorithmic C Data Types

The Catapult Algorithmic C data types provides designers with the ability to model bit-accuracy for integer, fixed-point, and floating-point data types, as well as to model streaming interfaces in an untimed C++ design. They also include a container class library for complex data types. The full documentation for the AC data types can be found at [AC DataTypes Reference manual](#).

1.2.1. ac_int: Integer Data Types

The ac_int bit-accurate data types allows modelling of an arbitrary precision signed or unsigned integer data type in C++.

Usage

In order to use the ac_int datatype the following file include should be used:

```
#include <ac_int.h>
```

The ac_int data types are templated for bit-width and signedness

```
ac_int<W,S> where W=bit width and S=signedness
```

Example usage:

```
ac_int<7, true> x;    // x is 7 bits signed
ac_int<19, false> y;  // y is 19 bits unsigned
```

Quantization and Overflow Behavior

The behavior of an ac_int data type is to truncate any fractional data during an assignment, and to “wrap” if the assigned data is greater than the precision (specified by W=width)

Example usage

```
ac_int<4,false> a = 1.5;  // value of "a" is 1
```

```
ac_int<4,false> a = 0x11; // value of "a" is 1, data has "wrapped"
```

Other Useful Data Type Methods

See the user docs for:

- `[]` – bit select operator for reading and writing bits of the data type
- `slc` – Read slice method for reading a bit slice from the data type
- `set_slc` – Write slice method for writing a bit slice to the data type
- `range` – method for reading/writing a bit slice from/to the data type
- `ac::log2_ceil<W>` – Statically compute `log2_ceil` of `W` bits
- `ac::nbits<W>` - Statically compute number of bits to count to `W`

1.2.2. `ac_fixed`: Fixed-Point Data Types

The `ac_fixed` bit-accurate data types allow modelling of an arbitrary precision signed or unsigned fixed-point data type in C++.

Usage

In order to use the `ac_fixed` datatype the following file include should be used:

```
#include <ac_fixed.h>
```

The `ac_fixed` data types are templated for bit-width, number of integer bits, and signedness

```
ac_fixed<W,I,S> where W=bit width, I=number of integer bits and S=signedness
```

Example usage:

```
ac_fixed<4,4,true> x; // bbbb. signed, 4 total bits, 4 integer bits  
ac_fixed<4,0,false> x; // .bbbb unsigned, 4 total bits, 4 fractional bits
```

Quantization and Overflow Behavior

Similar to the `ac_int` data types, the default behavior of `ac_fixed` data types is to truncate and wrap. However, the `ac_fixed` data types have two additional template parameters to control quantization and overflow behavior.

```
ac_fixed<int W, int I, bool S, ac_q_mode Q, ac_o_mode O>
```

where `Q` is the Quantization mode (responsible for rounding), and `O` is the overflow mode (responsible for saturation).

NOTE: If your design needs rounding and saturation you will get better results by using the built-in Quantization and Overflow modes rather than coding it explicitly in the source code.

Example

```
//4-bit fixed point integer with round to infinity and
//                                saturate to max of 15 and min of 0
ac_fixed<4,4,false,AC_RND,AC_SAT> a;

//stored value is 2
a = 1.5;

//stored value is 15
a = 20;
```

See the `ac_data` types manual for a full description of all Quantization and Overflow modes.

1.2.3. `ac_float`: Floating-Point Data Types

Catapult HLS supports a number of different [synthesizable floating point data types](#).

1.2.4. `ac_complex`: Complex Data Types

Catapult HLS supports a [complex class](#) that supports all of the bit-accurate as well as native data types.

1.2.5. Pretty-Printing Bit-Accurate Data Types

The Algorithmic C data types are templated classes. Inspecting the value of a data type variable in a debugger can be tedious due to having to push down into the class to find the actual data value. Because of this a pretty-printing python file is provided that can be used along with gdb(must support python). Thus, any debugger that uses gdb under-the-hood can enable pretty printing.

Add the following line to the `.gdbinit` file located in the user's home directory (File may need to be created):

Catapult Install

```
py exec(open("<Catapult InstallDir>/shared/pkg/ac_types/gdb/ac_pp.py").read())
```

Download from HLSLibs.org

```
py exec(open("<AC types gdb dir>/shared/pkg/ac_types/gdb/ac_pp.py").read())
```

1.3. Creating a Design Module with Interfaces

High-Level Synthesis for Untimed C++ creates Verilog modules from C++ classes.

1.3.1. Class-based Coding Style

High-Level Synthesis can synthesize a C++ class as a Verilog module if the class follows the following [coding style rules and use of pragmas](#). Otherwise, if no pragmas are used on the class it will be inlined during synthesis.

Coding Rules for Synthesizing a Class to a Module

- `#pragma hls_design top` tells HLS to make this the top-level design/module (This is not required and can be done as a synthesis constraint)
- The class constructor **MUST** be declared, even if it is empty. This is required by HLS and will generate an error otherwise
- The class must have a single public member function with a `#pragma hls_design interface`. This tells HLS to synthesize this member function as the design interface
- The class is free to have any other number of private data members and member functions

```
#pragma hls_design top
class simple {
private:
    <private data members>
    <private member functions>
public:

    simple() {}

    #pragma hls_design interface
    void run_simple(
        <interface variables>
    ) {
        <body>
    }
};
```

The previous C++ code will result in the following Verilog:

```
module simple(
    <Port list>
)
always@(posedge clk)
begin
    <module body>
end
endmodule
```

Catapult will synthesize the class public member function interface variables to module ports (Each variable can result in one or more ports depending on the interface protocol added during high-level synthesis). Also note that Catapult will synthesize the class to a synchronous process in RTL.

1.3.2. Data Interfaces

There are two types of data types that can be used for class public member function interface variable types; `ac_channel` data types for moving data into and out of the class, and any scalar data type (including user defined types) for configuration input data.

Streaming Interfaces (`ac_channel` class)

The `ac_channel` class is used to model streaming behavior in the untimed C++ design. This data type can be synthesized to a streaming hardware interface with simple data/ready/valid behavior. This coding style is based on Kahn Process Networks (https://en.wikipedia.org/wiki/Kahn_process_networks) and guarantees hardware to be synthesized that is functionally equivalent to the original C++. See the user documentation on [ac_channel](#) for additional information.

In order to use the `ac_channel` class library the following include is required:

```
#include <ac_channel.h>
```

The `ac_channel` class is templated for any data type, including user-defined data types.

```
ac_channel<ac_int<10,false> > data_in;
```

The `ac_channel` class has methods for reading and writing data:

```
T read()-Read a value of data type T from the channel
void write(T d)-write a value of type T into the channel
```

An `ac_channel` that is defined on the class public member function interface must be defined with a reference "&". This is because the `ac_channel` interface represents a hardware interface that is connecting to some other upstream or downstream module or FIFO.

The following code example shows the simple class defined before with `ac_channel` interfaces. This example reads(streams) 10 values from the input channel "din" and writes 10 values to the output channel "dout". It uses a "for" loop with 10 iterations to do this.

```
1
2 #include <ac_int.h>
3 #include <ac_channel.h>
4 #pragma hls_design top
5 class simple
6 {
7 public:
8
9     simple() {}
10
11     #pragma hls_design interface
12     void run_simple(ac_channel<ac_int<10,false> > &din,
13                   ac_channel<ac_int<10,false> > &dout) {
```

```
14     ac_int<10,false> data = din.read();
15     dout.write(data);
16 }
17 };
PATH: $MGC_HOME/shared/examples/docs/qstart/cxx/hier/class_hier_channel.h
```

Synthesizing an `ac_channel` to hardware results in a hardware interface with a data/ready/valid protocol. This hardware interface will “block” if the `ac_channel` read/write cannot complete. This has the result of stalling the synthesized hardware. However, the behavior is differing in software since you cannot stall the single threaded execution of the synthesizable design.

A read of an empty `ac_channel` in software will result in a runtime exception during execution of the C++. Therefore, the user must “guard” the reading of `ac_channel(s)` to prevent underflow in software simulation. This can be done using the channel “available(int N)” method

```
bool available(int N) - return true if there are N or more items in read channel
```

The *available* function MUST be used to guard the execution of the design. This has the effect of simulating blocking behavior. However, it may require modifying the design testbench to repeatedly call the public member function until sufficient data is available. The following code example shows how the “available” method prevents reading an empty channel.

```
1
2 #include <ac_int.h>
3 #include <ac_channel.h>
4 #pragma hls_design top
5 class simple
6 {
7 public:
8
9     simple() {}
10
11 #pragma hls_design interface
12 void run_simple(ac_channel< ac_int<10,false> > &din,
13               ac_channel< ac_int<10,false> > &dout) {
14     if (din.available(10)) {
15         for (int i=0; i<10; i++) {
16             ac_int<10,false> data = din.read();
17             dout.write(data);
18         }
19     }
20 }
21 };
PATH: $MGC_HOME/shared/examples/docs/qstart/cxx/hier/class_hier_available.h
```

The “available” method always synthesizes to *true* and has no effect on synthesis. Remember the synthesized hardware interface will “block” if data cannot be read, stalling the design.

Writing to an `ac_channel` in software simulation is never a problem as an `ac_channel` has infinite depth in software simulation. The synthesized hardware interface will “block” if data cannot be written, stalling the design.

Memory Interfaces

While HLS can directly [synthesize C++ arrays to memories](#), both internal and on the design interface,

ac_channel MUST be used for interface memories if the memory connects to another HLS design module. This allows HLS to automatically insert the required synchronization between modules/processes.

The following coding style rules describe how to code memory read interfaces using ac_channel. Following these rules will allow HLS to optimize away any local variables and not infer internal memories but just a memory interface.

- The ac_channel data type needs to be a struct containing an array

```
ac_channel<mem_struct> &mem;
```

NOTE: The struct that contains the array should only contain a single array and nothing else

NOTE: If multiple data fields are required, they need to be concatenated into a single bit-vector, and an array of that data type can be used inside the struct

NOTE: Using an array of structs will likely result in a scheduling failure for pipelined designs

- A local variable of the same struct type must be declared

```
mem_struct local_mem;
```

- The ac_channel interface is read once and copied into the local variable

```
mem_struct local_mem = mem.read();
```

- All operations must be performed on the local variable
- The local variable cannot be written
- The local variable cannot store any state between function calls

The following example shows a simple design with a Read Memory interface:

```
1
2 #include <ac_int.h>
3 #include <ac_channel.h>
4
5 struct mem_struct {
6     ac_int<10,false> data[1024];
7 };
8
9 #pragma hls_design top
10 class simple
11 {
12 public:
13
14     simple() {}
15
16     #pragma hls_design interface
17     void run_simple(ac_channel<mem_struct> &mem_in,
18                   ac_channel<ac_int<10,false> > &dout) {
```

```

19     if (mem_in.available(1)) {
20         mem_struct local_mem = mem_in.read();
21         for (int i=0; i<1024; i++) {
22             ac_int<10,false> data = local_mem.data[i];
23             dout.write(data);
24         }
25     }
26 }
27 };
PATH: $MGC_HOME/shared/examples/docs/qstart/cxx/hier/class_hier_mem_read_if.h

```

The following coding style rules describe how to code memory write interface using `ac_channel`. Following the rules will allow HLS to optimize away any local variables and not infer internal memories but just a memory interface

- The `ac_channel` data type needs to be a struct containing an array

```
ac_channel<mem_struct> &mem;
```

- A local variable of the same struct type must be declared

```
mem_struct local_mem;
```

- All operations must be performed on the local variable
- The local variable cannot be read
- The local variable is written once to the `ac_channel` interface

```
mem.write(local_mem);
```

- The local variable cannot store any state between function calls

The following example shows a simple design with a Write Memory interface:

```

1
2 #include <ac_int.h>
3 #include <ac_channel.h>
4
5 struct mem_struct {
6     ac_int<10,false> data[1024];
7 };
8
9 #pragma hls_design top
10 class simple
11 {
12 public:
13
14     simple() {}
15
16     #pragma hls_design interface
17     void run_simple(ac_channel<ac_int<10,false> > &din,
18                    ac_channel<mem_struct> &mem_out) {
19         if (din.available(1024)) {

```

```

20     mem_struct local_mem;
21     for (int i=0; i<1024; i++) {
22         ac_int<10,false> data = din.read();
23         local_mem.data[i] = data;
24     }
25     mem_out.write(local_mem);
26 }
27 }
28 };
PATH: $MGC_HOME/shared/examples/docs/qstart/cxx/hier/class_hier_mem_write_if.h

```

1.3.3. Configuration Interfaces

The previous sections showed how `ac_channel` should be used for moving data into and out of a design for both stream and memory interfaces. However, for the case where a design needs configuration inputs driven by external configuration status registers, a scalar variable should be used. HLS is able to optimize this type of input, which does not require any synchronization on the interface. Using “[direct input constraints](#)” on the interface allows HLS to remove any pipeline registers and directly drive wires from the interface into the synthesized data path. A “direct input” can also be specified in the C++ source using a pragma.

The following code example shows how to code a configuration input that is used to break out of a loop used for streaming data into and out of the design.

```

1
2 #include <ac_int.h>
3 #include <ac_channel.h>
4 #pragma hls_design top
5 class simple
6 {
7 public:
8     simple() {}
9
10    #pragma hls_design interface
11    void run_simple(ac_channel< ac_int<10,false> > &din,
12                  ac_channel< ac_int<10,false> > &dout,
13                  ac_int<4,false> &size) {
14        if (din.available(10)) {
15            for (int i=0; i<10; i++) {
16                ac_int<10,false> data = din.read();
17                dout.write(data);
18                if (i == size-1) {
19                    break;
20                }
21            }
22        }
23    }
24 };
PATH: $MGC_HOME/shared/examples/docs/qstart/cxx/hier/class_hier_direct.h

```

1.3.4. Copying Arrays from an Interface or Internally

The section on memory interfaces showed that a container class or struct is required for reading or writing arrays into an `ac_channel`. In general, it is a good idea to use a container class for copying arrays, either on the design interface or internal to the design. For convenience the AC data types includes an array class called `ac_array`.

The [ac_array class](#) is included as:

```
#include <ac_array.h>
```

The `ac_array` class is templated for the data type and number of elements:

```
template<typename T, int D0>
ac_array<int, 10> foo; // array if type integer with 10 elements
```

The `ac_array` class is overloaded to allow reading and writing array elements using the “`[]`” operator, and supports array dimensions up to 3 dimensions.

```
ac_array<int, 5, 10> foo; // array that is [5][10] elements
foo[0][0] = 1; // write element [0][0]
int d = foo[3][3]; // Read element [3][3];
```

The following example shows how the `ac_array` container class can be used.

```
1
2 #include <ac_int.h>
3 #include <ac_channel.h>
4 #include <ac_array.h>
5 #pragma hls_design top
6 class simple
7 {
8 public:
9     simple() {}
10    #pragma hls_design interface
11    #pragma hls_pipeline_init_interval 1
12    void run_block1(ac_channel<ac_array<ac_int<10,false>,10>> &din,
13                  ac_channel<ac_int<10,false>> &dout) {
14        ac_array<ac_int<10,false>,10> regs;
15        int acc = 0;
16        if (din.available(1)) {
17            regs = din.read(); // copy the array
18            for (int i=0; i<10; i++) {
19                acc += regs[i];
20            }
21            dout.write(acc);
22        }
23    }
24 };
PATH: $MGC_HOME/shared/examples/docs/qstart/cxx/hier/ac_array_example.h
```

1.4. Coding for Area

This section briefly covers the basics of how to code for smaller area. More advanced topics are discussed later when looking at coding for performance and how that can impact area. In general HLS is always performing automatic optimizations in order to try and reduce area, but coding style can have a big impact the final area. HLS has to synthesize the code as it's described, which may lead to unexpected larger area.

1.4.1. Mutually Exclusive Conditions

This topic is covered more in-depth in the [HLS Bluebook](#).

HLS will share resources inside of mutually exclusive conditions if area can be reduced. However, HLS may not detect mutual exclusivity if it is not explicit. The following example shows the case where HLS will not share the multiplier because the mutual exclusivity is not explicit.

```
// res0 and res1 are NOT shared
bool cond;
...
cond = <some expression>

if (cond)
    res0 = a * b;
...
if (!cond)
    res1 = c * d;
```

The following example rewrites the code so that the mutual exclusivity is explicit. HLS can now share the multiplier if it will save area.

```
// res0 and res1 are shared
bool cond;
...
cond = <some expression>

if (cond)
    res0 = a * b;
else
    res1 = c * d;
```

1.4.2. Making Things Constant

HLS is very good at optimizing constants. In many cases HLS does not understand the designer's intent and may "see" a variable as being dynamic. Using dynamic variables may result in larger area due to additional unneeded decode logic such as multiplexors and and/or logic. A good example is indexing into an array that has been mapped to registers. HLS can infer C++ arrays as either registers or memories. The example in

The following example shows how an array “shift_register” mapped to registers is indexed by a variable “sel”. Although “sel” can only take on values 0 and 15, the variable appears to be dynamic to HLS. Thus, HLS thinks that “sel” can index all 16 locations of “shift_register”. This leads to a 16 to 1 multiplexer being synthesized.

```
ac_int<8,false> shift_register[16]; // array mapped to registers
ac_int<4,false> sel; // Select variable to index into the shift_register array
ac_int<8,false> data;
...
<operate/write shift_register array>
...
sel = <compute sel> //sel can only be 0 or 15

data = shift_register[sel]
...
```

The following example shows how the previous code can be rewritten to use a constant to index the array. Because only “sel” values of 0 and 15 are possible the code compares “sel” against constant values and if the comparison is true, the array is indexed with that constant value. This will result in a two-input multiplexer compared to a 16-input multiplexor.

```
ac_int<8,false> shift_register[16]; // array mapped to registers
ac_int<4,false> sel; // Select variable to index into the shift_register array
ac_int<8,false> data;
...
<operate/write shift_register array>
...
sel = <compute sel>; // sel can only be 0 or 15

if (sel == 0)
    data = shift_register[0];
else if (sel==15)
    data = shift_register[15];
...
```

1.4.3. Loops

HLS synthesizes a “for” loop as a data-path along with an FSM to control it. The loop iterator is typically used to do a number of things such as index into an array, as part of an expression, or as part of a loop exit condition. If the loop bounds are known, HLS is able to optimize the loop counter as well as any control logic associated with the loop iterator. For a more in-depth discussion, refer to the [HLS Bluebook](#).

The code in Error: Reference source not found is a bad example of coding a loop for HLS. This design is intended to read from 1 sample to at most 1024 samples based on the interface variable “size”. However, the interface variable “size” is a 32-bit integer. If the loop exit condition is 32-bits, or is unknown, HLS cannot optimize the loop iterator, resulting in a 32-bit loop counter and 32-bit comparator logic for the exit condition.

Furthermore, the way the loop is coded would allow the loop to be skipped if size==0.

```

1
2 #include <ac_int.h>
3 #include <ac_channel.h>
4 #pragma hls_design top
5 class simple
6 {
7 public:
8     simple() {}
9
10    #pragma hls_design interface
11    void run_simple(ac_channel<ac_int<10,false> > &din,
12                  ac_channel<ac_int<10,false> > &dout,
13                  int size) {
14        if (din.available(1024)) {
15            for (int i=0; i<size; i++) {
16                ac_int<10,false> data = din.read();
17                dout.write(data);
18            }
19        }
20    }
21 };
PATH: $MGC_HOME/shared/examples/docs/qstart/cxx/loops/iterator_bad.h

```

A better way to code the previous example is to add a break condition at the end of the loop and use bit accurate data types for the loop iterator. The loop exit condition expression should also be cast to the same bit-accurate data type precision as the loop iterator. This will give the smallest area and is also needed for closing RTL code coverage. The following example recodes the previous example to use an unsigned bit-accurate data-type (ac_int<10,false>) for the loop iterator.

```

1
2 #include <ac_int.h>
3 #include <ac_channel.h>
4 #pragma hls_design top
5 class simple
6 {
7 public:
8     simple() {}
9
10    #pragma hls_design interface
11    void run_simple(ac_channel< ac_int<10,false> > &din,
12                  ac_channel< ac_int<10,false> > &dout,
13                  int size) {
14        if (din.available(1024)) {
15            for (ac_int<10,false> i=0; ; i++) {
16                ac_int<10,false> data = din.read();
17                dout.write(data);
18                if (i == ac_int<10,false>(size-1)) {
19                    break;
20                }
21            }
22        }
23    }
24 };
PATH: $MGC_HOME/shared/examples/docs/qstart/cxx/loops/iterator_good.h

```

In this example, a 10-bit unsigned integer can count from 0 to 1023, which gives a most 1024 loop iterations.

The loop exit condition (`i == size-1`) has been cast to `ac_int<10,false>` giving the minimum number of bits needed to count to 1023, leading to optimal comparator logic for the exit condition.

1.5. Coding for Performance and Area

HLS provides designers with the ability to add parallelism and control throughput using design constraints, as well as to allow multiple interconnected modules to run concurrently to one another. This coding style document does not cover the details of these HLS constraints and it is recommended to refer to the user documentation for a more in-depth discussion of [loop unrolling and loop pipelining](#).

1.5.1. Intra-Process Parallelism (Loop Unrolling)

HLS loop unrolling attempts to execute multiple loop iterations in parallel when synthesizing to RTL.

Loops can be unrolled using design constraints in the HLS tool or using pragmas in the source code. The loop unroll pragma is:

```
#pragma hls_unroll <number of iterations | yes>
```

Users can specify the number of times to unroll a loop or can use “yes” to fully unroll the loop. A loop must be bounded to fully unroll it.

Care must be taken when coding the loop body of an unrolled loop since it is easy to create dependent logic, resulting in longer latency, larger area, and longer HLS runtimes.

Avoid the following:

- Unrolled loops with conditional break
- Unrolled loops with conditional assignments
- Unrolled loops with saturating accumulators

The scenarios describe above can usually be avoided by rewriting the loop body code to be unconditional.

The following snippet shows a loop that is fully unrolled to accumulate the values of an array. The loop has a conditional break which results in a dependency chain of adders when the loop is unrolled. This means longer latency and larger area.

```
ac_int<13,false> acc = 0;
ac_int<10,false> data[16];
...
#pragma hls_unroll yes
for (int i=0; i<16; i+=1) {
    acc += data[i];
    if (i == size-1)
```

```
break;
}
```

The previous example can be rewritten to make the loop body unconditional. This coding style should always be used, if possible, when you know that the loop is to be unrolled. The loop exit condition “if(l == size-1)” and break are removed and the “?” operator is used to always accumulate, either “data[i]” or “0” if the conditional is false.

Making the loop body, or assignments in the loop body, unconditional will allow HLS to balance and optimize expressions. In this example it will result in a balanced adder tree.

```
ac_int<13,false> acc = 0;
ac_int<10,false> data[16];
...
#pragma hls_unroll yes
for (int i=0; i<16; i+=1) {
    acc += i <= size-1 ? data[i] : ac_int<10,false>(0);
}
```

NOTE: Catapult HLS users should always run Design Analyzer on the design and look at the Advisor view. This will identify many of these coding style issues!

1.5.2. Unrolling a Loop to Make the Loop Iterator Constant

It was shown earlier that HLS is able to optimize constants, leading to smaller area. This is especially true when dynamically indexing arrays mapped to registers inside of an unrolled loop.

Dynamically indexing writes to register arrays inside of unrolled loops is far more expensive than dynamically indexing reads. This is because unrolling the loop is creating multiple copies of the dynamically indexed writes to the register array. So, if possible, code should always be transformed so that the write indices are constant. Dynamic indexing of register array reads is much cheaper in terms of area.

These types of transformations can be done by taking advantage of the effects of loop unrolling on the loop iterator.

NOTE: A fully unrolled loop will result in the loop iterator becoming constant!

The following example shows how one array is copied to another using a variable “offset” to copy values to higher address locations than the input array. The variable “offset” is dynamic (or programmable) which results in multiple copies of the dynamic writes created due to the loop being fully unrolled. This will lead to larger area and longer runtimes when synthesized in HLS.

```
1
2 #include <ac_int.h>
3 #pragma hls_design top
4 void dynamic_index_bad(uint8 data_in[20],
5                       uint5 offset_in,
6                       uint8 data_out[20])
7 {
8     uint8 data[20];
9     uint8 data_off[20];
```

```

10  uint5 offset = offset_in;
11  #pragma unroll yes
12  for (int i=0; i<20; i++) {
13      data[i] = data_in[i];
14      data_off[i] = 0;
15  }
16  #pragma unroll yes
17  for (int i=0; i<20; i++) {
18      if (i+offset<20) {
19          data_off[i + offset] = data[i];
20      }
21  }
22
23  #pragma unroll yes
24  for (int i=0; i<20; i++) {
25      data_out[i] = data_off[i];
26  }
27 }
PATH: $MGC_HOME/shared/examples/docs/qstart/cxx/loops/dynamic_index_bad.cpp

```

The following example shows how the previous design was rewritten to make the write indices to “data_off” constant. An additional loop has been added that is fully unrolled and compares the value of the loop iterator “j” to “j == offset”. The array “data_off” is written using the iterators “i+j” when the condition “j==offset && j+i<20”. The index “j+i” becomes constant when both loops are fully unrolled, resulting in constant indices used for the register array writes. This results in a very efficient hardware structure.

```

1  #include <ac_int.h>
2  void dynamic_index_good(uint8 data_in[20],
3                          uint5 offset_in,
4                          uint8 data_out[20])
5  {
6  {
7      uint8 data[20];
8      uint8 data_off[20];
9      uint5 offset = offset_in;
10 #pragma unroll yes
11  for (int i=0; i<20; i++) {
12      data[i] = data_in[i];
13      data_off[i] = 0;
14  }
15
16 #pragma unroll yes
17  for (int i=0; i<20; i++) {
18 #pragma unroll yes
19      for (int j=0; j<20; j++) {
20          if (j==offset && j+i < 20) {
21              data_off[i + j] = data[i];
22          }
23      }
24  }
25
26 #pragma unroll yes
27  for (int i=0; i<20; i++) {
28      data_out[i] = data_off[i];
29  }
30 }
PATH: $MGC_HOME/shared/examples/docs/qstart/cxx/loops/dynamic_index_good.cpp

```

1.5.3. Achieving Maximum Throughput

Loop pipelining is used to tune the throughput of a design. Most HLS scheduling failures are due to the loop pipelining constraints being impossible to achieve. These failures are typically related to the use of bandwidth limiting resource inside of a loop such as `ac_channel` IO reads/writes and/or memory reads/writes. Please refer to the User Docs for an in-depth discussion of loop pipelining.

Loop Pipelining pragma

Loop pipelining constraints can be applied as a tool directive or as a pragma in the source code.

The loop pipelining pragma takes the form:

```
#pragma hls_pipeline_init_interval <1,2,3,etc, where 1 is max pipelining>
```

It was shown previously that loops could be used to control data movement by using `ac_channel` to stream data into an out of a design. This coding style combined with loop pipelining can allow designers to get the desired throughput.

The example shown in Error: Reference source not found shows how the “for” loop that reads and writes `ac_channel` interfaces is pipelined with `II=1`. This means that the synthesized design will be able to read and write once per clock cycle. It should be noted that the pipelining pragma could also have been applied to the function call itself for the maximum performance. In this example the “for” loop will ramp-up and ramp-down as it is entered and exited.

```
1
2 #include <ac_int.h>
3 #include <ac_channel.h>
4
5 #pragma hls_design interface
6 void run_simple(ac_channel<ac_int<10,false>> &din,
7               ac_channel<ac_int<10,false>> &dout)
8 {
9     #pragma hls_pipeline_init_interval 1
10    for (ac_int<4,false> i=0; i<10; i+=1) {
11        ac_int<10,false> data = din.read();
12        dout.write(data);
13    }
14 }
15
PATH: $MGC_HOME/shared/examples/docs/qstart/cxx/loops/pipeline_pragma.h
```

Figure 1 Loop Pipelining Pragma

Limit reads/writes to `ac_channel` or Arrays mapped to Memories

The minimum achievable pipeline initiation interval is dependent on the number of bandwidth limiting resources inside a loop body. A pipelining constraint that cannot be satisfied will result in a [scheduling failure](#) during HLS synthesis.

The following example shows an example design that accumulates every two values read from the `ac_channel` “din” and has a pipeline constraint with the initiation interval equal to 1 (`II=1`). This design will fail

to schedule because there are 2 reads to the `ac_channel` interface “din” inside of the loop body. At best it could be pipelined with `II=2`.

```

1
2 #include <ac_int.h>
3 #include <ac_channel.h>
4 #pragma hls_design interface
5 #pragma hls_pipeline_init_interval 1
6 void run_simple(ac_channel<ac_int<10,false>> &din,
7                ac_channel<ac_int<10,false>> &dout)
8 {
9     if (din.available(1024)) {
10         for (ac_int<10,false> i=0; i<1024; i+=2) {
11             ac_int<10,false> acc = 0;
12             acc = din.read();
13             acc += din.read();
14             dout.write(acc);
15         }
16     }
17 }
18
PATH: $MGC_HOME/shared/examples/docs/qstart/cxx/loops/pipeline_bad.h

```

The following example shows how to rewrite the previous design example to allow pipelining with `II=1`. The reading of “din” has been reduced to one read per loop iteration. Additional code has been added to ensure that every two reads are accumulated and written out.

```

1
2 #include <ac_int.h>
3 #include <ac_channel.h>
4
5 ac_int<10,false> acc; //data member is persistent, initialized to 0
6 #pragma hls_design interface
7 #pragma hls_pipeline_init_interval 1
8 void run_simple(ac_channel<ac_int<10,false>> &din,
9                ac_channel<ac_int<10,false>> &dout)
10 {
11     if (din.available(1024)) {
12         for (ac_int<10,false> i=0; i<1024; i+=1) {
13             acc += din.read(); // read once per loop iteration
14             if (i&1) { // write and clear on even loop iterations
15                 dout.write(acc);
16                 acc = 0;
17             }
18         }
19     }
20 }
21
PATH: $MGC_HOME/shared/examples/docs/qstart/cxx/loops/pipeline_good.h

```

1.5.4. Inter-Process Concurrency

Complex designs may require multiple concurrent processes to meet performance goals or to manage complexity.

Sequential Loop Behavior

One of the most common reasons for partitioning a design into multiple processes is when the design contains multiple dependent sequential loops. Fully dependent (second loop depends on data from the first loop after it completes) sequential loops will execute sequentially in hardware, leading to lower performance.

The following example shows a pipelined design that contains sequential loops. The first loop writes an array mapped to registers in ascending order. The second loop reads the register array in descending order. This design will take ~20 clock cycles to execute in hardware because the second loop must wait for the first loop to finish executing. These loops should be pushed into separate classes so that they can be synthesized as concurrent processes to improve performance. Doing this requires coding the design to interconnect the separate processes using `ac_channel`.

```

1
2 #include <ac_int.h>
3 #include <ac_channel.h>
4 #pragma hls_design top
5 class simple
6 {
7 public:
8
9     simple() {}
10
11     #pragma hls_design interface
12     #pragma hls_pipeline_init_interval 1
13     void run_simple(ac_channel<ac_int<10,false> > &din,
14                    ac_channel<ac_int<10,false> > &dout) {
15         ac_int<10,false> regs[10];
16         if (din.available(10)) {
17             for (int i=0; i<10; i++) {
18                 regs[i] = din.read();
19             }
20             for (int i=0; i<10; i++) {
21                 dout.write(regs[9-i]); // reverse order
22             }
23         }
24     }
25 };
PATH: $MGC_HOME/shared/examples/docs/qstart/cxx/loops/seq_dependent.h

```

Multiple Interconnected Processes

The following coding style rules must be adhered to in order to synthesize a design with multiple interconnected processes

1. A process is coded as a class with a single public member function with `#pragma hls_design interface` to define the process interface
2. Class interface variable data types should use `ac_channel` for data interfaces and can use scalar variable types for top-level configuration status register data
3. A parent class should be used to create instances of child classes synthesized as concurrent

processes

- The parent class should have a single public member function with `#pragma hls_design` interface
- The public member function should be used to interconnect the children
- The `ac_channel` streaming class MUST be used interconnect the child classes
- Only class instances and `ac_channel` data members should be defined in the parent class
- The parent class should only be used to interconnect child classes. There cannot be any “glue” logic added where the classes are interconnected. In other words, the parent class should only be calling the public member functions of the child classes

The code in Error: Reference source not found shows how the example in Error: Reference source not found has been rewritten based on the rules for coding multiple-concurrent processes.

This example design has been coded to do the following:

- Classes `block0` and `block1` have been created for each of the sequential loops from the example in the previous section.
- The `block0` class does the following:
 - Defines a local variable “regs” of type `ac_array` that has 10 elements to represent the register array
 - Has a loop to read the input channel and write data into the register array
 - Writes the register array `ac_array` variable to an `ac_channel` of the same type on the design interface
- The `block1` class does the following:
 - Reads data from an `ac_channel` interface of type `ac_array` into a local `ac_array` “regs”
 - Has a loop that reads from the “regs” array in reverse order and writes to an `ac_channel` output interface
- The top block shows how the two previous classes are “stitched” together to create a multi-process design. The top block does the following:
 - Defines instances of `block0` and `block1`, `block0_inst` and `block1_inst`, classes as private data members of the class `multi_process`
 - Defines a private data member “connect” of type `ac_array` to interconnect the `block0` and `block1` instances
 - Defines a public member function that interconnects the two class instances by calling the

run_block0 and run_block1 member functions and using the ac_channel data member to interconnect them.

```

1
2 #include <ac_int.h>
3 #include <ac_channel.h>
4 #include <ac_array.h>
5
6 class block0
7 {
8 public:
9     block0() {}
10    #pragma hls_design interface
11    #pragma hls_pipeline_init_interval 1
12    void run_block0(ac_channel<ac_int<10,false>> &din,
13                  ac_channel<ac_array<ac_int<10,false>,10>> &dout) {
14        ac_array<ac_int<10,false>,10> regs;
15        if (din.available(10)) {
16            for (int i=0; i<10; i++) {
17                regs[i] = din.read();
18            }
19            dout.write(regs);
20        }
21    }
22 };
23
24 #pragma hls_design top
25 class block1
26 {
27 public:
28     block1() {}
29     #pragma hls_design interface
30     #pragma hls_pipeline_init_interval 1
31     void run_block1(ac_channel<ac_array<ac_int<10,false>,10>> &din,
32                   ac_channel<ac_int<10,false>> &dout) {
33         ac_array<ac_int<10,false>,10> regs;
34         if (din.available(1)) {
35             regs = din.read();
36             for (int i=0; i<10; i++) {
37                 dout.write(regs[9-i]); // reverse order
38             }
39         }
40     }
41 };
42
43 #pragma hls_design top
44 class multi_process_top
45 {
46     block0 block0_inst;
47     block1 block1_inst;
48     ac_channel<ac_array<ac_int<10,false>,10>> connect;
49 public:
50     multi_process_top() {}
51     #pragma hls_design interface
52     #pragma hls_pipeline_init_interval 1
53     void run_simple(ac_channel<ac_int<10,false>> &din,
54                   ac_channel<ac_int<10,false>> &dout) {
55         block0_inst.run_block0(din,connect);
56         block1_inst.run_block1(connect,dout);

```

```
57 }
58 };
PATH: $MGC_HOME/shared/examples/docs/qstart/cxx/loops/seq_hier.h
```

Shared Memory Interconnect

A shared memory interconnect between two processes can be created by following the coding style rules covered in the section on Error: Reference source not found. The memory interfaces described in Error: Reference source not found Error: Reference source not found be interconnected using an `ac_channel` that has the same “`mem_struct`” data type. This is shown in Error: Reference source not found. This coding style will allow Catapult to infer a shared memory interconnect, either a single memory or a ping-pong memory, to connect the two processes. The type of memory interconnect can be controlled during synthesis.

```
class shared_mem{
    write_mem block0;
    read_mem block1;
    ac_channel<mem_struct> mem_connect;
public:
    shared_mem() {}

#pragma hls_design interface
    void run(ac_channel<ac_int<10,false> > &din,
            ac_channel<ac_int<10,false> > &dout) {
        block0.run(din,mem_connect);
        block1.run(mem_connect,dout);
    }
};
```

Figure 2 Shared Memory Interconnect

1.5.5. Coding Memory Architecture

High-Level Synthesis synthesizes the architecture described in the C++ design. Multiple micro-architectures can be generated from this C++ description using different synthesis constraints, but HLS cannot add an underlying memory architecture that is not described in the source.

One of the most common reasons for sub-optimal performance is due to a lack of memory architecture described in the C++. Poorly described, or not described at all, [memory architectures tend to limit loop pipelining and unrolling optimizations](#).

A very simple example of this is a moving average filter that averages 3 sequential data values from an array.

```
1
2 #ifndef __MOVING_AVG_1D__
3 #define __MOVING_AVG_1D__
4 #include <ac_fixed.h>
5 #include "types.h"
6 class moving_average_1d_algorithm
7 {
8     int clip(int idx) {
```

```

9     if (idx < 0) {
10         return 0;
11     } else if (idx > 99) {
12         return 99;
13     } else {
14         return idx;
15     }
16 }
17 public:
18     moving_average_1d_algorithm() {}
19
20     #pragma hls_design interface
21     void run(ac_fixed<8,8,false> din[100], ac_fixed<8,8,false> dout[100]) {
22         ac_fixed<12,8,false> acc = 0;
23         AVG: for (int i=0; i<100; i++) {
24             acc = din[clip(i-1)]*k[0] + din[i]*k[1] + din[clip(i+1)]*k[2];
25             dout[i] = acc;
26         }
27     }
28 };
29
30 #endif
PATH: $MGC_HOME/shared/examples/docs/qstart/cxx/memory/moving_average_1d_algorithm.h

```

The problem with the implementation shown in the previous design example is that mapping the “din” array to a single port memory will prevent pipelining the design with II=1. This is because pipelining with II=1 implied starting a new iteration of the AVG loop every clock cycle. This means reading 3 values from “din” in parallel every clock cycle, which is impossible if “din” has a single memory read port.

The key to designing efficient memory architectures that can be optimized efficiently by HLS is to code the C++ memory architecture so that the internal bandwidth of the C++ matches the bandwidth of limiting resources such as memories, memory interfaces, or IO. In this example the code needs to be rewritten to reduce the reading of “din” to once per loop iteration of the AVG loop.

Analysis of the code in Error: Reference source not found that each loop iteration reuses two of the values read in the previous iteration. Storing these two values (shift register storage) locally allows the design to be rewritten so that “din” is only read once per loop iteration. This type of architecture is known as a “Sliding Window” architecture. This is shown in the following design example:

```

1
2 #ifndef __MOVING_AVG_1D_ARCH__
3 #define __MOVING_AVG_1D_ARCH__
4 #include <ac_fixed.h>
5 #include "types.h"
6 class moving_average_1d_architecture
7 {
8
9 public:
10     moving_average_1d_architecture() {}
11
12     #pragma hls_design interface
13     void run(ac_fixed<8,8,false> din[100], ac_fixed<8,8,false> dout[100]) {
14         ac_fixed<12,8,false> acc = 0;
15         ac_fixed<8,8,false> dat0,dat1,dat2;
16         ac_fixed<8,8,false> win0,win1,win2;
17         AVG: for (int i=0; i<101; i++) {
18             if (i != 100) {

```

```

19     dat0 = din[i];
20 }
21 // boundary
22 if (i==1) {
23     win2 = dat1;
24 } else {
25     win2 = dat2;
26 }
27 if (i==100) {
28     win0 = dat1;
29 } else {
30     win0 = dat0;
31 }
32 win1 = dat1;
33
34 acc = win2*k[0] + win1*k[1] + win0*k[2];
35 // Shift register
36 dat2 = dat1;
37 dat1 = dat0;
38 if (i != 0) {
39     dout[i-1] = acc;
40 }
41 }
42 }
43 };
44
45 #endif
PATH: $MGC_HOME/shared/examples/docs/qstart/cxx/memory/moving_average_1d_architec-
ture.h

```

Figure 3 1-D Moving Average Memory Architecture

The previous moving average example showed that a shift-register architecture could be used to store a “window” of data for a 1-d array input to achieve higher performance by being able to pipeline with II=1. This same approach can be used for the slightly more complicated example when the input is a 2-d array and the moving average filter averages data across different rows on the input image. This algorithm is shown in the following code example:

```

1
2 #ifndef __MOVING_AVG_2D_ALG__
3 #define __MOVING_AVG_2D_ALG__
4 #include <ac_fixed.h>
5 #include "types.h"
6 class moving_average_2d_algorithm
7 {
8     int clip(int idx) {
9         if (idx < 0) {
10             return 0;
11         } else if (idx > 99) {
12             return 99;
13         } else {
14             return idx;
15         }
16     }
17 public:
18     moving_average_2d_algorithm() {}
19
20     #pragma hls_design interface

```

```

21 void run(ac_fixed<8,8,false> din[100][100],
22         ac_fixed<8,8,false> dout[100][100]) {
23     ac_fixed<12,8,false> acc = 0;
24     ROW: for (int i=0; i<100; i++) {
25         COL: for (int j=0; j<100; j++) {
26             acc = din[clip(i-1)][j]*k[0] +
27                 din[i][j]*k[1] +
28                 din[clip(i+1)][j]*k[2];
29             dout[i][j] = acc;
30         }
31     }
32 }
33 };
34
35 #endif
PATH: $MGC_HOME/shared/examples/docs/qstart/cxx/memory/moving_average_2d_algorithm.h

```

The 2-D moving average algorithm will have the same type of performance bottleneck as the 1-D example when “din” is mapped to a single port memory. Pipelining with $II=1$ will not be possible because there are 3 reads of “din” for each loop iteration of the COL loop. The main difference between the 2-D and 1-D moving average algorithms is that the 2-D case is reading from 3 different rows of data for each iteration of the “COL” loop. This means that a shift-register sliding window architecture is not the solution. The desired architecture needs to buffer multiple rows of data in order to be able to achieve the best-case performance. This means coding a line-buffer sliding window memory architecture where separate internal memories are used to store row data from the input array. This is essentially a line-buffer sliding window memory architecture and is shown in Error: Reference source not found. This code rewrites adds two arrays line_buf0 and line_buf1, and the row data from “din” is shifted through the arrays. The C++ is structured so that the arrays are only read or written once per COL loop iteration, which allows them to be mapped to a memory with one read port and one write port.

```

1
2 #ifndef __MOVING_AVG_2D_ARCH__
3 #define __MOVING_AVG_2D_ARCH__
4 #include <ac_fixed.h>
5 #include "types.h"
6 class moving_average_2d_architecture
7 {
8 public:
9     moving_average_2d_architecture() {}
10
11     #pragma hls_design interface
12     void run(ac_fixed<8,8,false> din[100][100],
13             ac_fixed<8,8,false> dout[100][100]) {
14         ac_fixed<12,8,false> acc = 0;
15         ac_fixed<8,8,false> dat0,dat1,dat2;
16         ac_fixed<8,8,false> win0,win1,win2;
17         ac_fixed<8,8,false> line_buf0[100];
18         ac_fixed<8,8,false> line_buf1[100];
19         ROW: for (int i=0; i<101; i++) {
20             COL: for (int j=0; j<100; j++) {
21                 if (i != 100) {
22                     dat0 = din[i][j];
23                 }
24                 dat2 = line_buf1[j];
25                 dat1 = line_buf0[j];
26                 // boundary
27                 if (i==1) {

```

```

28     win2 = dat1;
29     } else {
30     win2 = dat2;
31     }
32     if (i==100) {
33     win0 = dat1;
34     } else {
35     win0 = dat0;
36     }
37     win1 = dat1;
38
39     acc = win2*k[0] + win1*k[1] + win0*k[2];
40     // Shift memories
41     line_buf1[j] = line_buf0[j];
42     line_buf0[j] = dat0;
43     if (i != 0) {
44     dout[i-1][j] = acc;
45     }
46     }
47 }
48 }
49 };
50
51 #endif
PATH: $MGC_HOME/shared/examples/docs/qstart/cxx/memory/moving_average_2d_architec-
ture.h

```

Figure 4 2-D Moving Average Memory Architecture

1.6. Coding for Power

Just like coding for area, designers need to be explicit about the architecture described in the C++ when power consumption is a consideration. This means structuring the code so that switching is minimized when possible. When coding for power users need to think about how the design is going to be optimized in Catapult HLS. Are loops being unrolled or left rolled? Are arrays being mapped to memories or registers? Understanding how the design will be synthesized, and what the expected hardware structure is, should help determine how to structure the C++ code. Catapult is able to analyze and [optimize for power](#) and coding style will have a large impact on the results.

A simple set of examples that illustrates this are discussed below.

The first example is a programmable, parallel multiply-accumulate that is fed by a parallel shift register “regs”, where the “regs” array is mapped to registers. In this example all loops are unrolled and the number of multiply-accumulates is programmable based on an interface variable “size”. This design is not efficient for power consumption as it always is shifting all 31 elements of the register “regs” regardless of the value of “size”, which determines the number of multiply-accumulates.

```

1
2 #include "parallel_shift.h"
3
4 void power(ac_channel<int> &din,
5           int coeffs[31],
6           uint6 size,
7           ac_channel<int> &dout)
8 {

```

```

9  static int regs[31];
10 int acc = 0;
11 #pragma unroll yes
12 for (int i=30; i>=0; i--) {
13     if (i==0)
14         { regs[i] = din.read(); }
15     else
16         { regs[i] = regs[i-1]; }
17 }
18 #pragma unroll yes
19 for (int i=0; i<31; i++) {
20     acc += (i<size) ? regs[i]*coeffs[i] : 0;
21 }
22 dout.write(acc);
23
24 }
PATH: $MGC_HOME/shared/examples/docs/qstart/cxx/lowPower/parallel_shift.cpp

```

A more power efficient way to code the previous example is to gate the shifting of the data based on the “size” configuration input. The following code shows this improved design.

```

1
2 #include "parallel_shift.h"
3
4 void power_opt(ac_channel<int> &din,
5               int coeffs[31],
6               uint6 size,
7               ac_channel<int> &dout)
8 {
9     static int regs[31];
10    int acc = 0;
11
12    #pragma unroll yes
13    for (int i=30; i>=0; i--) {
14        if (i==0)
15            { regs[i] = din.read(); }
16        else if (i<size-1)
17            { regs[i] = regs[i-1]; }
18    }
19    #pragma unroll yes
20    for (int i=0; i<31; i++) {
21        acc += (i<size) ? regs[i]*coeffs[i] : 0;
22    }
23    dout.write(acc);
24 }
PATH: $MGC_HOME/shared/examples/docs/qstart/cxx/lowPower/parallel_shift_opt.cpp

```

The previous two examples show how switching can be minimized by gating off the shift register based on the configuration input “size”. However, in these examples, the multiply-accumulate loop was fully parallelized. Let’s look at the case where the multiply-accumulate loop is left rolled. The following example shows the un-optimized version of this implementation.

```

1
2 #ifndef __ROTATE__
3 #define __ROTATE__
4 #include <ac_int.h>
5 #include <ac_channel.h>
6
7 void power(ac_channel<int> &din,

```

```

8      int coeffs[31],
9      uint6 size,
10     ac_channel<int> &dout)
11 {
12     static int regs[31];
13     int acc = 0;
14 #pragma unroll yes
15     for (int i=30; i>=0; i--) {
16         if (i==0) {
17             regs[i] = din.read();
18         } else {
19             regs[i] = regs[i-1];
20         }
21     }
22
23     for (int i=0; i<31; i++) {
24         acc += regs[i]*coeffs[i];
25         if (i==size-1) { break; }
26     }
27     dout.write(acc);
28 }
29 #endif
PATH: $MGC_HOME/shared/examples/docs/qstart/cxx/lowPower/rotate.h

```

The previous example slightly different than initial parallel shift implementation. The multiply-accumulate loop has been written with a conditional break based on “size” that allows the loop to exit early. This is more efficient since the loop is left “rolled”. However, this implementation is not power efficient since the shift register “regs” shifts all elements for each new data value read from “din”. A better implementation is to use a circular buffer implementation for the shift register so that only one element of “regs” is ever written for each new value of “din”, reducing the switching, as shown in the following example.

```

1
2 #ifndef __ROTATE_OPT__
3 #define __ROTATE_OPT__
4 #include <ac_int.h>
5 #include <ac_channel.h>
6
7 void power_opt(ac_channel<int> &din,
8               int coeffs[31],
9               uint6 size,
10              ac_channel<int> &dout)
11 {
12     static int regs[31];
13     static uint5 idx_wr=0,idx_rd=0;
14     int acc = 0;
15
16     //Circular Buffer
17     regs[idx_wr] = din.read();
18     idx_wr++;
19     if (idx_wr == 31) {
20         idx_wr = 0;
21     }
22
23     for (int i=0; i<31; i++) {
24         idx_rd = idx_wr;

```

```

25     acc += regs[idx_rd]*coeffs[i];
26     if (idx_rd == 0) {
27         idx_rd = 30;
28     } else {
29         idx_rd--;
30     }
31     if (i==size-1) { break; }
32 }
33 dout.write(acc);
34 }
35 #endif
PATH: $MGC_HOME/shared/examples/docs/qstart/cxx/lowPower/rotate_opt.h

```

This same circular buffer approach can also be used when designing more complicated memory architectures that require memories to buffer intermediate data.

1.7. Using HLS Math Libraries

High-level synthesis is used to turn algorithms written in C++ into high-quality RTL. One of the “sweet spots” for HLS is for compute intensive algorithms in communications, image and video processing, and machine learning, to name a few. Many of the algorithms in these domains require math functions such as square root, arc tangent, sine, cosine, etc. These functions are available in the standard C++ `math.h` library. However, Catapult HLS does not directly synthesize `math.h`. Instead, it provides highly optimized synthesizable versions of the most commonly used functions in `math.h`. These synthesizable versions are implemented using piecewise linear, CORDIC and recursive algorithms for a range of area/performance tradeoffs. These libraries are available both in the Catapult install tree as well as from hlslibs.org and support math functions for bit-accurate integer, fixed-point and floating-point types.

To use the catapult math libraires [<link to math docs>](#) you will need the following include paths

- Compiler include path for Catapult’s include directory

```
<Catapult Install Dir>/Mgc_home/shared/include
```

- Includes added to the C++ source

```
#include <math/mgc_ac_math.h>
#include <ac_math.h>
```

The following example demonstrates using the synthesizable `math.h` library for integer square root.

```

1
2 #include <ac_int.h>
3 #include <ac_math.h>
4 #pragma hls_design top
5 void test_math(ac_int<10,false> din, ac_int<6,false> &dout)
6 {
7     ac_math::ac_sqrt(din,dout);
8 }
PATH: $MGC_HOME/shared/examples/docs/qstart/cxx/math/test_math.h

```

Chapter 2: Coding with SystemC

This quick start guide is meant to give designers a jump-start for writing SystemC code, with/without MatchLib, for High-Level Synthesis.

SystemC allows designers to model both timed, loosely timed, and untimed behavior. This is very powerful as it enables a designer to work at multiple levels of abstraction within the same design, modeling at the pin-level where needed, as well as keeping code abstract to allow the HLS tool freedom to optimize the hardware. Modeling hardware just using SystemC is usually written at the signal level and has some limitations in terms of the types of designs that can be implemented. Typically, these designs must read and write the IO every clock cycle and use a valid bit to control the flow of data. Wait statements are often used to control when (on which clock edge) a signal is read/written, and the use of multiple wait statements in the same process will prevent throughput-accurate simulation. Furthermore, precisely describing designs with backpressure is not possible with this coding style.

The MatchLib SystemC library overcomes the limitations of signal-level design by providing users with a library of interfaces that enable throughput-accurate modeling, including modeling of backpressure.

Both signal-level and MatchLib coding styles are covered in the document, along with the general concepts of coding for HLS. However, this document is by no means comprehensive and more in-depth coverage of coding style and general HLS optimizations can be found in the HLS Blue Book as well as the Catapult User Reference Manual.

2.1. Compiling Designs with SystemC and MatchLib Include Files

Designing in SystemC for HLS will require including both library header files and SystemC pre-compiled libraries. This requires that the C++ compiler know where to find these header files.

The typical header files that will need to be included are:

```
// SystemC
#include <systemc.h>

// MatchLib
#include <mc_connections.h>

// AC bit-accurate data types
#include <ac_int.h>
#include <ac_fixed.h>
```

The libraries needed for SystemC compilation are *systemc* and *pthread*

There are two possible locations where the header files and libraries can be included and depends on

whether the user has the Catapult HLS tool installed on their network.

2.1.1. Including Header Files from the Catapult Install Tree

If Catapult HLS is installed on the user's network an environment variable should be set to point to the Mgc_home directory in the install tree

```
setenv CATAPULT_HOME <Path to Catapult install>/Mgc_home
```

Then designs can be compiled by including the Catapult include directory:

SystemC Designs:

```
g++ -I $CATAPULT_HOME/shared/include -L $CATAPULT_HOME/shared/lib \  
-lsystemc -lpthread <design file name>.cpp
```

SystemC Designs with MatchLib:

```
g++ -DCONNECTIONS_ACCURATE_SIM -DSC_INCLUDE_DYNAMIC_PROCESS \  
-I $CATAPULT_HOME/shared/include -L $CATAPULT_HOME/shared/lib \  
-lsystemc -lpthread <design file name>.cpp
```

Where CONNECTIONS_ACCURATE_SIM enables throughput-accurate modeling for MatchLib designs.

MatchLib also has a fast simulation mode that can be enabled by defining CONNECTIONS_FAST_SIM instead of CONNECTIONS_ACCURATE_SIM

2.1.2. Including Header Files after Downloading from HLSLibs.org

A Catapult install is not required to use SystemC or MatchLib Connections. SystemC and MatchLib Connections can both be downloaded separately. SystemC 2.3.3 is required.

https://github.com/hlslibs/matchlib_connections

<https://www.accellera.org/downloads/standards/systemc>

2.1.3. Setting Compiler Optimization Levels

Compiler optimizations will affect the execution runtime of the users C++ program

Debugging C++ Designs

Compiler optimizations should be disabled when designing and debugging using the bit-accurate data types. This will allow designers to step through the code in a C++ debugger such as VSCode or Eclipse. Optimizations can be disabled by setting the optimization level to zero (-O0)

```
g++ -O0 <all other args>
```

Fast Runtime

If the user wants the fastest execution time, they should set the optimization level to `-O3`

```
g++ -O3 <all other args>
```

2.1.4. Using the Algorithmic C Data Types

The Catapult Algorithmic C data types provides designers with the ability to model bit-accuracy for integer, fixed-point, and floating-point data types, as well as to model streaming interfaces in an untimed C++ or SystemC design. They also include a container class library for complex data types. The SystemC data types have the same functionality as the AC datatypes. However the SystemC data types require choosing between `sc_int`, `sc_uint`, `sc_biguint`, etc, based on the precision of the integer data type. Also, the SystemC fixed-point data types simulate far slower than the AC fixed-point data types so it is recommended that AC data-types are used in SystemC designs.

The full documentation for the AC data types can be found at
`$MGC_HOME/shared/pdffdocs/ac_datatypes.pdf`

2.1.5. `ac_int`: Integer Data Types

The `ac_int` bit-accurate data types allows modeling of an arbitrary precision signed or unsigned integer data type in C++.

Usage

In order to use the `ac_int` datatype the following file include should be used:

```
#include <ac_int.h>
```

The `ac_int` data types are templated for bit-width and signedness

```
ac_int<W,S> where W=bit width and S=signedness
```

Example usage:

```
ac_int<7, true> x; // x is 7 bits signed
ac_int<19, false> y; // y is 19 bits unsigned
```

Quantization and Overflow Behavior

The behavior of an `ac_int` data type is to truncate any fractional data during an assignment, and to “wrap” if the assigned data is greater than the precision (specified by `W=width`)

Example usage

```
ac_int<4,false> a = 1.5; //value of "a" is 1
ac_int<4,false> a = 0x11; // value of "a" is 1, data has "wrapped"
```

Other Useful Data Type Methods

See the user docs for:

- `[]` – bit select operator for reading and writing bits of the data type
- `slc` – Read slice method for reading a bit slice from the data type
- `set_slc` – Write slice method for writing a bit slice to the data type
- `range` – method for reading/writing a bit slice from/to the data type
- `ac::log2_ceil<W>` – Statically compute log2_ceil of W bits
- `ac::nbits<W>` - Statically compute number of bits to count to W

2.1.6. `ac_fixed`: Fixed-Point Data Types

The `ac_fixed` bit-accurate data types allow modeling of an arbitrary precision signed or unsigned fixed-point data type in C++.

Usage

In order to use the `ac_fixed` datatype the following file include should be used:

```
#include <ac_fixed.h>
```

The `ac_fixed` data types are templated for bit-width, number of integer bits, and signedness

```
ac_fixed<W,I,S> where W=bit width, I=number of integer bits and S=signedness
```

Example usage:

```
ac_fixed<4,4,true> x; // bbbb. signed, 4 total bits, 4 integer bits  
ac_fixed<4,0,false> x; // .bbbb unsigned, 4 total bits, 4 fractional bits
```

Quantization and Overflow Behavior

Similar to the `ac_int` data types, the default behavior of `ac_fixed` data types is to truncate and wrap. However, the `ac_fixed` data types have two additional template parameters to control quantization and overflow behavior.

```
ac_fixed<int W, int I, bool S, ac_q_mode Q, ac_o_mode O>
```

where Q is the Quantization mode (responsible for rounding), and O is the overflow mode (responsible for saturation).

NOTE: If your design needs rounding and saturation you will get better results by using the built-in Quantization and Overflow modes rather than coding it explicitly in the source code.

Example

```
ac_fixed<4,4,false,AC_RND,AC_SAT> a;//4-bit fixed point integer with round to
infinity and saturate to max of 15 and min of 0
    a = 1.5;//stored value is 2
    a = 20;//stored value is 15
```

See the `ac_data` types manual for a full description of all Quantization and Overflow modes.

2.1.7. `ac_float`: Floating-Point Data Types

Catapult HLS supports a number of different synthesizable floating point data types. The documentation can be found at `$MGC_HOME/shared/pdftdocs/ac_math_ref.pdf`

2.1.8. `ac_complex`: Complex Data Types

Catapult HLS supports a complex class that supports all of the bit-accurate as well as native data types.

2.1.9. Pretty-Printing Bit-Accurate Data Types

The Algorithmic C data types are templated classes. Inspecting the value of a data type variable in a debugger can be tedious due to having to push down into the class to find the actual data value. Because of this a pretty-printing python file is provided that can be used along with gdb(must support python). Thus, any debugger that uses gdb under-the-hood can enable pretty printing.

Add the following line to the `.gdbinit` file located in the user's home directory (File may need to be created):

Catapult Install

```
py exec(open("<Path to Catapult Mgc_home dir>/shared/pkg/ac_types/gdb/ac_pp-
p.py").read())
```

Download from HLSLibs.org

```
py exec(open("<Path to AC types gdb dir>/shared/pkg/ac_types/gdb/
ac_pp.py").read())
```

2.2. Creating a Design Module with Interfaces

2.2.1. Understanding How HLS Synthesizes a SystemC Design

High-Level Synthesis of SystemC creates Verilog modules and one or more concurrent or combinational processes.

HLS “sees” a SystemC design as a set of processes (`SC_THREAD`, `SC_METHOD`) communicating only through signals (`sc_signal`). HLS synthesis occurs on each process one at a time, in complete isolation and no analysis/optimizations are performed across processes. This allows huge designs to be synthesized with HLS, as long as each process is not too large. Each synthesized combinational process (`SC_METHOD`)

becomes combinational logic in RTL, and each sequential process becomes exactly one FSM + sequential datapath logic in RTL.

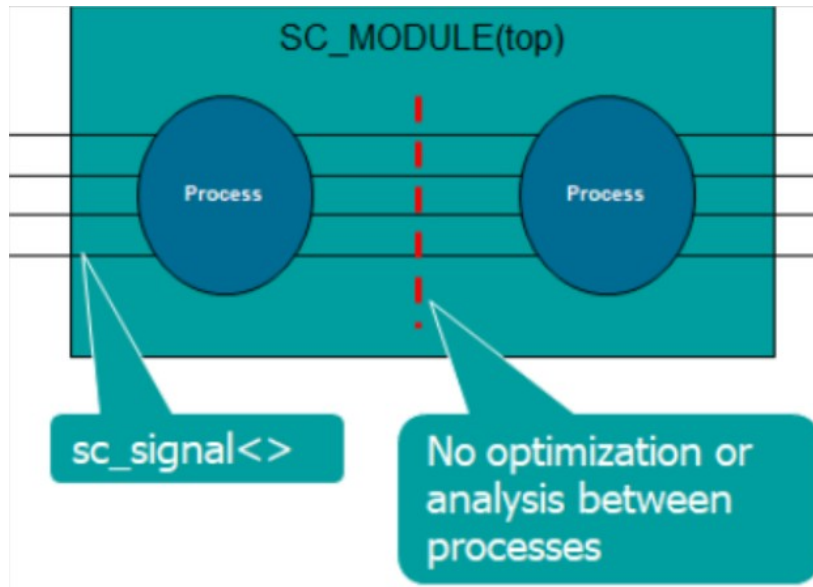


Figure 5: Processes Interconnected by Signals

Primary optimizations done by HLS are in construction of the FSM and Datapath (e.g. resource sharing). Unlike RTL synthesis, HLS is (usually) allowed to add clock cycles (latency/pipelining) into design to improve quality of results; e.g. loops are often pipelined to maintain throughput while latency is added by HLS. Properly constructed models and proper usage of HLS should show no functional differences (aside from latency differences) pre and post HLS. For SystemC HLS, reset behaviors and IO protocols are specified in the pre-HLS model and synthesized into RTL together with rest of model

2.2.2. Coding SystemC Modules

Defining a C++ class that inherits from the SystemC `sc_module` class will be synthesized as a Verilog module. The SystemC module declaration consists of

- Class that inherits from `sc_module`
- Ports of type `sc_in` and `sc_out`
- A module constructor
 - Initializes variables
 - Defines clock or input sensitivity
 - Defines one or more module threads
 - Defines module resets, can have both synchronous and asynchronous resets

- One or more threads that implement either a sequential or combinational process

```
#include<systemc.h>
class simple_module : public sc_module{
public:
    <module ports>

    SC_CTOR(simple_module)
    {
        <Threads, resets, clock, etc.>
    }

    <Thread implementations>
};
```

The previous code example will result in the following Verilog:

```
module simple_module(<Port list>);
    <sequential or combinational process>
endmodule
```

2.2.3. Sequential Processes

A sequential process is created by defining an SC_THREAD in the module constructor along with the thread implementation in the class. The clock sensitivity/polarity and type of reset(s) are defined in the constructor as well.

The following code example shows an sc_module with a single sequential process. Note the following:

- “ac_sysc_macros.h” provides the convenience macro CCS_INIT_S1 for naming the module ports
- The ports are defined using sc_in and sc_out along with the port data type
- The module constructor defines an SC_THREAD called run
- The “run” thread is sensitive to the positive edge of the clock
- The “run” thread has both asynchronous and synchronous resets
 - Async reset is active low
 - Sync reset is active high
- The “run” thread implementation implements a synchronous process that reads “din” and writes it to “dout”
- The code up to the first “wait()” statement in “run” is the reset state of the hardware and resets “dout” to zero

- The “run” thread has an infinite “while” loop that implements the sequential process
- The infinite “while” loop MUST have a “wait()” statement in the loop body or else the process will hang

```

1
2 #include <systemc.h>
3 #include <ac_sysc_macros.h>
4 class simple_seq_module : public sc_module
5 {
6 public:
7     sc_in<bool>          CCS_INIT_S1(clk);
8     sc_in<bool>          CCS_INIT_S1(rstn);
9     sc_in<bool>          CCS_INIT_S1(srst);
10    sc_in<unsigned short> CCS_INIT_S1(din);
11    sc_out<unsigned short> CCS_INIT_S1(dout);
12
13    SC_CTOR(simple_seq_module) {
14        SC_THREAD(run);
15        sensitive << clk.pos();
16        async_reset_signal_is(rstn, false);
17        reset_signal_is(srst, true);
18    }
19
20    void run() {
21        //reset state
22        dout.write(0);
23        wait();
24        while (1) { //clocked process
25            dout.write(din.read());
26            wait(); //required wait
27        }
28    }
29 };
PATH: $MGC_HOME/shared/examples/docs/qstart/sysc/interfaces/module_seq.h

```

2.2.4. Combinational Processes

Combinational processes can be created using an SC_METHOD that is sensitive to all of its inputs. This is shown below in Figure 6 where:

- An “SC_METHOD” is declared in the module constructor
- The SC_METHOD is sensitive to all of the module inputs

The method implementation implements the combinational process, in this case it is just a wire connecting “dout” to “din”

```

1
2 #include <ac_sysc_macros.h>
3 class simple_module : public sc_module
4 {
5 public:
6     sc_in<unsigned short> CCS_INIT_S1(din);
7     sc_out<unsigned short> CCS_INIT_S1(dout);
8
9     SC_CTOR(simple_module) {
10         SC_METHOD(run);

```

```
11     sensitive << din;  
12 }  
13  
14 void run() {  
15     dout.write(din.read());  
16 }  
17 };  
PATH: $MGC_HOME/shared/examples/docs/qstart/sysc/interfaces/module_combo.h
```

2.2.5. Data Interfaces and Processes

There are two design styles used in coding SystemC designs, signal-level design in SystemC, and coding MatchLib Connections designs. Both of these design styles have their own advantages and disadvantages. These coding styles are related to the types of interfaces used in the design.

There are essentially three types of interfaces for moving data in/out of a module or process; signal interfaces using a valid only protocol, MatchLib Connections Interfaces, and instantiated memory interfaces. While using MatchLib interfaces is generally recommended because of its throughput-accurate modeling capabilities and ease-of-use, signal interfaces using a signal-level SystemC coding style can be used in certain designs, especially designs that don't require back-pressure.

Signal-Level SystemC Coding Style

The rules for coding signal-level SystemC are:

- Use Catapult macros for naming ports

```
#include <ac_sysc_macros.h>
```

- Clock and reset use `sc_in<bool>`
- All data IO uses `sc_in<>`, `sc_out<>`
- Code up to first `wait()` in process models the reset state

Every `sc_out` should be assigned reset value here

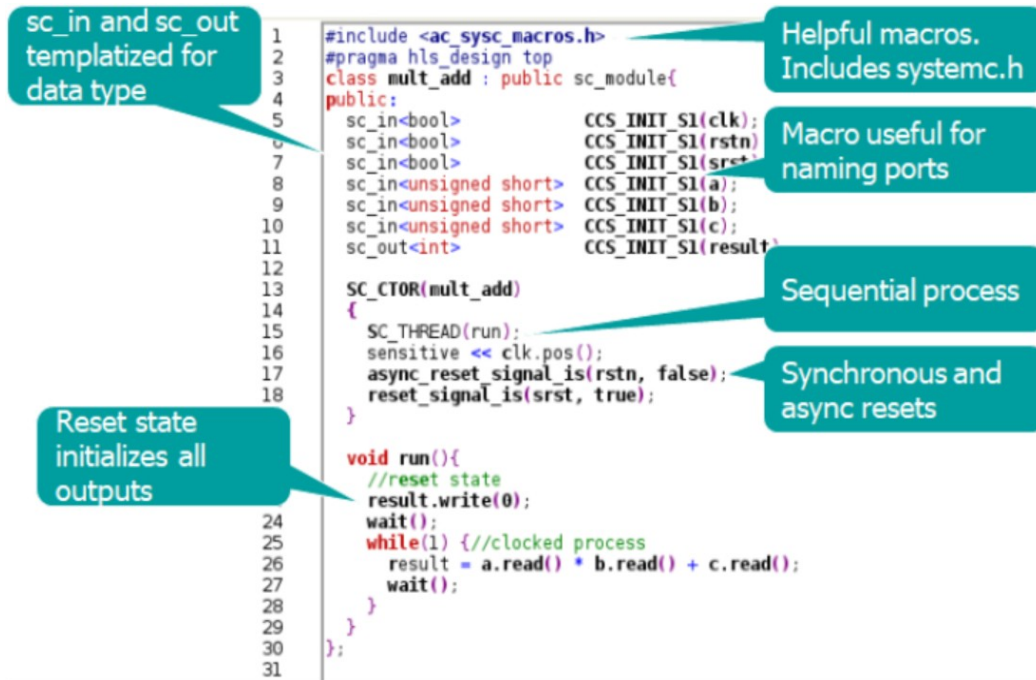


Figure 6: RTL in SystemC

- Every loop requires a `wait()` statement (unless fully unrolled)
- Catapult will add an implicit `wait()` statement if needed



Figure 7: Required Wait Statement

By default,

- Signal reads occur at preceding wait statement in SC source
- Signal writes take effect at next wait statement in SC source

- Catapult is free to insert scheduler states (pipeline registers) between SC wait statements
- Use IO_MODE=fixed if you don't want Catapult to add any states
- Keep in mind that SystemC testbench and DUT are modeling hardware, not software
 - e.g., shared variable between 2 processes in testbench or DUT may cause races
 - Use sc_signal, MatchLib Connections, Connections::FifoModule<T,N>, etc., to avoid races

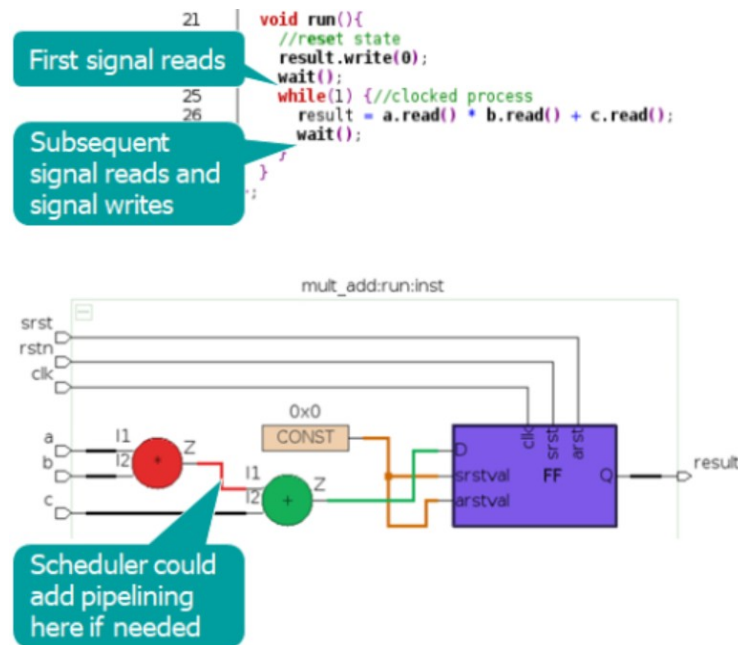


Figure 8: Signal Reads and Writes

Some of the added value over hand-coding RTL is:

- Scheduling of the data path
- Catapult loop pipelining still works
- Any AC + SC datatype can be used (ac_fixed, ac_float, etc.).
- Any AC function can be used (e.g., AC Math)
- Module hierarchy and IO in RTL will be the same as in SystemC

Signal Interfaces Using a Valid-only Protocol

Signal-level design in SystemC can be done reliably if:

- The hardware is constrained so that the inputs are guaranteed to be read every clock cycle. This implies that the hardware must be able to consume the input data whenever its valid
- A valid (single bit) input is used to indicate when the input data is valid. This valid bit must guard the thread body from executing if no valid data is available
- A valid (single bit) output is used to indicate when the output data is valid
- Multiple inputs with separate valid bits are guaranteed to be scheduled on the same clock edge by the High-level Synthesis tool

```

1
2 #include <systemc.h>
3 class valid_protocol : public sc_module
4 {
5 public:
6     sc_in<bool>          CCS_INIT_S1(clk);
7     sc_in<bool>          CCS_INIT_S1(rstn);
8     sc_in<bool>          CCS_INIT_S1(srst);
9     sc_in<bool>          CCS_INIT_S1(vld_in);
10    sc_in<unsigned short> CCS_INIT_S1(din);
11    sc_out<unsigned short> CCS_INIT_S1(dout);
12    sc_out<bool>          CCS_INIT_S1(vld_out);
13
14    SC_CTOR(valid_protocol) {
15        SC_THREAD(run);
16        sensitive << clk.pos();
17        async_reset_signal_is(rstn, false);
18        reset_signal_is(srst, true);
19    }
20
21    void run() {
22        //reset state
23        dout.write(0);
24        vld_out.write(0);
25        unsigned int acc = 0;
26        wait();
27        while (1) { //clocked process
28            if (vld_in.read()) {
29                acc += din.read();
30                dout.write(acc);
31            }
32            vld_out.write(vld_in.read());
33            wait();
34        }
35    }
36 };
PATH: $MGC_HOME/shared/examples/docs/qstart/sysc/interfaces/signal_valid_protocol.h

```

The previous example shows a coding example that implements a valid-only protocol. The example does the following:

- Implements a synchronous process (SC_THREAD) that accumulates the input “din”
- Resets the valid output “vld_out” and accumulator “acc” in the reset action

- Only executes the thread body when “vld_in” is true
- Copies “vld_in” to “vld_out”

MatchLib Connections Interfaces

MatchLib is a synthesizable SystemC library of commonly used functions and components such as interfaces, FIFOs, interconnect, etc. It allows most design and verification of SoCs to be performed in SystemC, verifying system-level performance earlier, and simulates 30x faster than RTL for timing-accurate simulation. MatchLib enables control-oriented design and provides easy-to-use throughput accurate modeling.

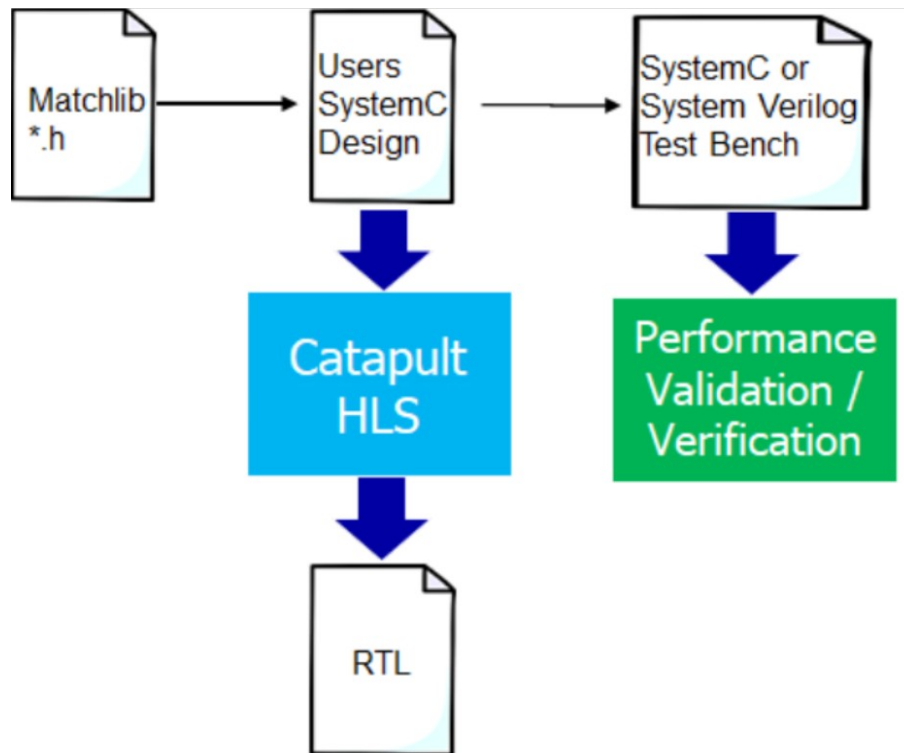


Figure 9: MatchLib Library

MatchLib Connections is a library and API of latency-insensitive channels. All components of this library are synthesizable using HLS. The Connections library consists of ports and channels that implement a data/ready/valid protocol. Connections are templated for the data type T.

Table 1: MatchLib Connections

Type	Name	Description
Port	In<T>	In port with Pop() and PopNB() methods
Port	Out<T>	Out port with Push() and PushNB() methods
Channel	Combinational<T>	Combinationally connects ports with Pop(), PopNB(), Push(), and PushNB() methods

MatchLib Connections provides both blocking (Push() and Pop()) and non-blocking (PushNB() and PopNB()) IO.

It is recommended that blocking IO should always be used when possible as it makes integration and debug

much simpler. The SW simulation and Hardware will stall when a Pop/Push call blocks. Push and Pop IO transactions complete when ready and valid are both high.

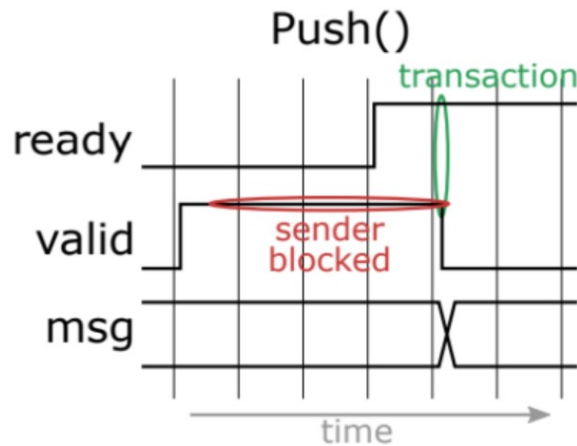


Figure 10: MatchLib Blocking Push

The rules for using MatchLib Connections are simple and straightforward:

- Include the connections library

```
#include <mc_connections.h>
```

- Include helper macros

CCS_INIT_S1 for port/module constructor naming

- Consider enabling pipeline flushing to allow outputs to flush when there are no more inputs

This only affects synthesis

- Add In and Out connections ports for the design
- Connections must be reset in the reset state of the thread
 - Call *.Reset(); method
 - Error generated in pre-hls simulation if reset is not set

The following example shows a design that performs a multiply-add on three Connections inputs, a, b, and c, and writes the result to a Connections output. Note that the Connections IO are all reset in the reset action. This design will simulate with a throughput of one output produced per clock cycle.

```
1
2 #include <ac_sysc_macros.h>
3 #include <mc_connections.h>
4 #pragma hls_design top
```

```

5 class mult_add : public sc_module
6 {
7 public:
8     sc_in<bool> CCS_INIT_S1(clk);
9     sc_in<bool> CCS_INIT_S1(rstn);
10
11     Connections::In <ac_int<16,false>> CCS_INIT_S1(a);
12     Connections::In <ac_int<16,false>> CCS_INIT_S1(b);
13     Connections::In <ac_int<16,false>> CCS_INIT_S1(c);
14     Connections::Out<ac_int<33,false>> CCS_INIT_S1(result);
15
16     SC_CTOR(mult_add) {
17         SC_THREAD(main);
18         sensitive << clk.pos();
19         async_reset_signal_is(rstn, false);
20     }
21
22 private:
23     void main() {
24         a.Reset();
25         b.Reset();
26         c.Reset();
27         result.Reset();
28         wait();
29         #pragma hls_pipeline_init_interval 1
30 #pragma pipeline_stall_mode flush
31         while (1) {
32             ac_int<16> ai = a.Pop();
33             ac_int<16,false> bi = b.Pop();
34             ac_int<16,false> ci = c.Pop();
35             ac_int<33,false> psum = ai*bi + ci;
36             result.Push(psum);
37         }
38     }
39 };
PATH: $MGC_HOME/shared/examples/docs/qstart/syssc/interfaces/matchlib_blockio.h

```

Synchronization Interfaces

MatchLib Connections also provides interfaces for synchronizing one or more processes to incoming data. Synchronization is typically required when the design/process has either configuration or memory inputs, which contain no explicit rdy/vld protocol, or for signaling another process, or external hardware such as a CPU or arbiter, to indicate that data has been processed and is available. Synchronization interfaces must be read/written outside of a pipelined data path as they are usually not pipelineable. Because of this, synchronization interfaces should be used on designs where the pipelined data path is processing larges amounts of data between synchronization events, minimizing the couple cycle overhead of the synchronization interface. Synchronization interfaces have only rdy/vld signaling and pass no data.

Type	Name	Description
Port	SyncIn	In port with SyncPop() and SyncPopNB() methods
Port	SyncOut	Out port with SyncPush() and SyncPushNB methods
Channel	SyncChannel	Combinationally connects ports with SyncPop() SyncPopNB(), SyncPush(), and SyncPushNB() methods

The Connections synchronization interfaces will be demonstrated when discussing memory and configuration interfaces.

Memory Interfaces

Catapult HLS allows C++ arrays to be mapped to memories or registers. This works for both C++ and SystemC synthesis when the array is local to a process. However, for SystemC, arrays mapped to memories on a module/process interface, or arrays between processes, are not supported for synthesis. This is because Catapult SystemC synthesis requires `sc_in/sc_out` interfaces. Therefore, instantiated SystemC memories are used, where the memory model provides a pin-level implementation for synthesis. This pin-level SystemC memory model is created by running the Catapult Memory Generator.

SystemC memories consist of a set of C++ classes for instantiating memories and memory interfaces. The memory interfaces consist of just the address, data, and control to interface to the memory read and write ports, and the SystemC memories contain the actual storage for the memory as well as ports for reading and writing.

The example shown below illustrates a simple design with a single memory read port on the top-level design interface. The memory locations are read, accumulated, and written to a Connections output port.

```

1
2 #include "DPRAM.h"
3 class mem_add : public sc_module
4 {
5 public:
6     sc_in<bool> CCS_INIT_S1(clk);
7     sc_in<bool> CCS_INIT_S1(rstn);
8     DPRAM_model<>::rd_port<ac_int<8>,32> mem;
9     Connections::Out<ac_int<13>> CCS_INIT_S1(data_out);
10    SC_CTOR(mem_add) {
11        SC_THREAD(run);
12        sensitive << clk.pos();
13        async_reset_signal_is(rstn, false);
14    }
15    void run() {
16        mem.reset();
17        data_out.Reset();
18        wait();
19        #pragma hls_pipeline_init_interval 1
20    #pragma pipeline_stall_mode flush
21        while (1) {
22            ac_int<13> acc = 0;
23            RD_MEM:for (int i = 0; i < 32; i++) {
24                ac_int<8> tmp = mem[i];
25                acc = acc + tmp;
26                #ifndef __SYNTHESIS__
27                wait();
28                #endif
29            }
30            data_out.Push(acc);
31        }
32    }
33 };
PATH: $MGC_HOME/shared/examples/docs/qstart/sysc/interfaces/mem_simple.h

```

Some of the key details of the previous example are:

- Lines 1 and include the MatchLib library and the SystemC memory model
- Line 8 instantiates a memory read port, that is an input to the design.
 - DPRAM_model is the name of the memory class created from the Catapult Memory Generator
 - rd_port is the name assigned to the SystemC memory read interface that reads data from a SystemC memory
 - The read port memory interface is templated for the data type and number of words. In this case ac_int<8> and 32 words.
- Line 16 reset the memory interface. This reset call is required.
- Line 24 the memory interface is read using the “[]” operator. There are also read/write methods provided with the memory model
- Lines 26-28 are for SystemC simulation only. For throughput-accurate simulation of the design, the memory uses an untimed model. Thus, a wait is explicitly added for simulation only.

One of the problems with the following memory example is that the memory interface can immediately be read once the design comes out of reset. There is no way to know if the external memory that the interface connects to has been written. Therefore, a synchronization interface is needed to control when the external memory can be read by the memory interface. This could be a SyncIn type interface or a Connections::In interface depending on the design. The example below shows how SyncIn can be used to control when the memory is read:

```

1
2 #include "DPRAM.h"
3 class mem_add : public sc_module
4 {
5 public:
6   sc_in<bool> CCS_INIT_S1(clk);
7   sc_in<bool> CCS_INIT_S1(rstn);
8   Connections::SyncIn CCS_INIT_S1(sync);
9   DPRAM_model<>::rd_port<ac_int<8>,32> mem;
10  Connections::Out<ac_int<13>> CCS_INIT_S1(data_out);
11  SC_CTOR(mem_add) {
12    SC_THREAD(run);
13    sensitive << clk.pos();
14    async_reset_signal_is(rstn, false);
15  }
16  void run() {
17    sync.Reset();
18    mem.reset();
19    data_out.Reset();
20    wait();
21    while (1) {
22      ac_int<13> acc = 0;
23      sync.SyncPop();
24      #pragma hls_pipeline_init_interval 1
25      #pragma pipeline_stall_mode flush
26      RD_MEM:for (int i = 0; i < 32; i++) {
27        ac_int<8> tmp = mem[i];
28        acc = acc + tmp;

```

```

29     #ifndef __SYNTHESIS__
30     wait();
31     #endif
32     }
33     data_out.Push(acc);
34     }
35 }
36 };
PATH: $MGC_HOME/shared/examples/docs/qstart/syssc/interfaces/mem_syncin.h

```

Some of the important details of this example are:

- Line 8 defines a Connections SyncIn input
- Line 17 resets the SyncIn interface in the reset action
- Line 23 reads the SyncIn interface by calling the SyncPop method. The hardware will stall here until SyncPop completes. The external memory would be loading here prior to the SyncPop completing. Note that the pipelining has been moved to the nested loops as SyncPop/SyncPush are not pipelinable.

2.2.6. Configuration Interfaces

The previous sections showed how MatchLib Connection can be used for streaming data into and out of a design. However, many designs can have inputs that are intended to be used to configure the hardware. These inputs are often driven by external configuration status registers that have been programmed by a CPU that is part of the larger SoC system. These configuration inputs have no explicit synchronization and are considered pseudo-static in the sense that they do not change value while the data-path is running. This allows HLS to optimize these types of inputs to remove any internal pipeline registers resulting from reading the configuration inputs.

Configuration inputs should be coded using the “sc_in<T>” port type along with a #pragma to tell Catapult to remove any associated internal pipeline registers with that interface. The pragma is “#pragma hls_direct_input” and should be defined right before the port declaration.

Synchronization is usually needed for these types of designs since toggling the configuration input while the data-path is active can result in corrupting the data that is “in-flight”.

The following example shows how to code a configuration input that is used to break out of a loop used for streaming data into and out of the design. Note the following:

- Lines 10 and 11, a configuration input “num_madds” is defined to control the number of multiply-accumulates the hardware will perform. It is defined to be a “direct_input”

The design uses a SyncIn, lines 9 and 22, to ensure that the configuration input is setup and help stable before the data-path begins running

- Line 30, the configuration input is used to break out of the loop.

```

1
2 #include <mc_connections.h>
3 class mult_add : public sc_module

```

```

4 {
5 public:
6   sc_in<bool> CCS_INIT_S1(clk);
7   sc_in<bool> CCS_INIT_S1(rstn);
8   Connections::In <ac_int<16, false>> CCS_INIT_S1(a);
9   Connections::In <ac_int<16, false>> CCS_INIT_S1(b);
10  Connections::SyncIn CCS_INIT_S1(sync);
11  #pragma hls_direct_input
12  sc_in<ac_int<6, false>> CCS_INIT_S1(num_madds);
13  Connections::Out<ac_int<38, false>> CCS_INIT_S1(result);
14  SC_CTOR(mult_add) {
15    ...
16  }
17 private:
18  void main() {
19    ... //reset action
20    wait();
21    while (1) {
22      ac_int<38, false> acc = 0;
23      sync.SyncPop();
24      #pragma hls_pipeline_init_interval 1
25  #pragma pipeline_stall_mode flush
26      for (int i=0; i<32; i++) {
27        ac_int<16> ai = a.Pop();
28        ac_int<16, false> bi = b.Pop();
29        acc += ai*bi;
30        printf("%d \n", num_madds.read().to_int());
31        if (i==num_madds.read()-1)
32          { break; }
33      }
34      result.Push(acc);
35    }
36  }
37 };
PATH: $MGC_HOME/shared/examples/docs/qstart/syssc/interfaces/config_interface.h

```

2.2.7. Handling User-defined Structs

It is typical that designs may wish to pass multiple data in parallel as an atomic operation. To do this, users create a custom struct that can contain multiple data members, arrays, etc.

Both MatchLib and SystemC require some additional member functions and member data to be added to user defined structs in order to compile. MatchLib needs to know the sum of all of the widths of the struct data members and requires a “Marshall” function implementation that concatenates the data into a single bitvector. Both SystemC and MatchLib need the struct to implement the “sc_trace” function and the stream “<<” operator. This is shown below for a struct that contains 2 data fields “a” and “b”

```

1
2 #include <ac_int.h>
3 struct my_data_struct {
4   ac_int<16, false> a;
5   ac_int<16, false> b;
6
7   //Matchlib only
8   static const unsigned int width = 16+16;
9   //Matchlib only
10  template <unsigned int Size> void Marshall(Marshaller<Size> &m) {

```

```

11     m &a;
12     m &b;
13 }
14 //SystemC and Matchlib
15 inline friend void sc_trace(sc_trace_file *tf, const my_data_struct &v,
16                             const std::string &NAME ) {
17     sc_trace(tf,v.a,  NAME + ".a");
18     sc_trace(tf,v.b,  NAME + ".b");
19 }
20 //SystemC and Matchlib
21 inline friend std::ostream &operator<<(ostream &os,
22                                         const my_data_struct &rhs) {
23     os << rhs.a << " ";
24     os << rhs.b << " ";
25     return os;
26 }
27 };
PATH: $MGC_HOME/shared/examples/docs/qstart/syssc/interfaces/user_structs.h

```

2.3. Coding for Area

This section briefly covers the basics of how to code for smaller area. More advanced topics are discussed later when looking at coding for performance and how that can impact area. In general HLS is always performing automatic optimizations in order to try and reduce area, but coding style can have a big impact the final area. HLS has to synthesize the code as it's described, which may lead to unexpected larger area.

2.3.1. Mutually Exclusive Conditions

This topic is covered more in-depth in the HLS Bluebook

HLS will share resources inside of mutually exclusive conditions if area can be reduced. However, HLS may not detect mutual exclusivity if it is not explicit. The following example shows the case where HLS will not share the multiplier because the mutual exclusivity is not explicit.

```

bool cond;
...
cond = <some expression>

if(cond)
    res0 = a * b;
...
if(!cond)
    res1 = c * d;

```

The following code snippet shows how the previous code can be rewritten so that the mutually exclusivity is explicit. HLS can now share the multiplier if it will save area.

```

bool cond;
...
cond = <some expression>

```

```

if(cond)
    res0 = a * b;
else
    res1 = c * d;

```

2.3.2. Making Things Constant

HLS is very good at optimizing constants. In many cases HLS does not understand the designer's intent and may "see" a variable as being dynamic. Using dynamic variables may result in larger area due to additional unneeded decode logic such as AND-OR logic and multiplexors. A good example is indexing into an array that has been mapped to registers. HLS can infer C++ arrays as either registers or memories. The following example shows how an array "shift_register" mapped to registers is indexed by a variable "sel". Although "sel" can only take on values 0 and 15, the variable appears to be dynamic to HLS. Thus, HLS thinks that "sel" can index all 16 locations of "shift_register". This leads to a 16 to 1 multiplexer being synthesized.

```

ac_int<8,false> shift_register[16]; //array mapped to registers
ac_int<4,false> sel; //Select variable to index into the shift_register array
ac_int<8,false> data;
...
<operate/write shift_register array>
...
sel = <compute sel> //sel can only be 0 or 15

data = shift_register[sel]
...

```

Error: Reference source not found shows how the code from Error: Reference source not found can be rewritten to use a constant to index the array. Because only "sel" values of 0 and 15 are possible the code compares "sel" against constant values and if the comparison is true, the array is indexed with that constant value. This will result in a two-input multiplexer compared to a 16-input multiplexer.

```

ac_int<8,false> shift_register[16]; //array mapped to registers
ac_int<4,false> sel; //Select variable to index the shift_reg array
ac_int<8,false> data;
...
<operate/write shift_reg array>
...
sel = <compute sel> //sel can only be 0 or 15

if(sel == 0)
    data = shift_reg[0];
else if(sel==15)
    data = shift_reg[15];
...

```

2.3.3. Loops

HLS synthesizes a “for” loop as a data-path along with an FSM to control it. The loop iterator is typically used to do a number of things such as index into an array, as part of an expression, or as part of a loop exit condition. If the loop bounds are known, HLS is able to optimize the loop counter as well as any control logic associated with the loop iterator. For a more in-depth discussion see the HLS Bluebook.

The following code snippet is a bad example of coding a loop for HLS. This design is intended to read from 1 sample to at most 1024 samples based on the interface variable “size”. However, the interface variable “size” is a 32-bit integer. If the loop exit condition is 32-bits, or is unknown, HLS cannot optimize the loop iterator, resulting in a 32-bit loop counter and 32-bit comparator logic for the exit condition. Furthermore, the way the loop is coded would allow the loop to be skipped if size==0.

```
for(int i=0; i<size;i++){
    ac_int<10,false> data = din.Pop();
    dout.write(data);
}
```

A better way to code this loop is to add a break condition at the end of the loop and use bit accurate data types for the loop iterator. The loop exit condition expression should also be cast to the same bit-accurate data type precision as the loop iterator. This will give the smallest area and is also needed for closing RTL code coverage.

The following example shows how the previous example has been recoded to use an unsigned bit-accurate data-type (ac_int<10,false>) for the loop iterator. A 10-bit unsigned integer can count from 0 to 1023, which gives a most 1024 loop iterations. The loop exit condition (i == size-1) has been cast to ac_int<10,false> giving the minimum number of bits needed to count to 1023, leading to optimal comparator logic for the exit condition.

```
...
for(ac_int<10,false> i=0; ;i++){
    ac_int<10,false> data = din.Pop();
    dout.write(data);
    if(i == ac_int<10,false>(size-1))
        break;
}
...
}
```

2.4. Coding for Performance and Area

HLS provides designers with the ability to add parallelism and control throughput using design constraints, as well as to allow multiple interconnected modules to run concurrently to one another. This coding style document does not cover the details of these HLS constraints and it is recommended to refer to the user documentation for a more in-depth discussion of loop unrolling and loop pipelining.

2.4.1. Intra-Process Parallelism (Loop Unrolling)

HLS loop unrolling attempts to execute multiple loop iterations in parallel when synthesizing to RTL.

Loops can be unrolled using design constraints in the HLS tool or using pragmas in the source code. The loop unroll pragma is:

```
#pragma unroll <number of iterations | yes>
```

Users can specify the number of times to unroll a loop or can use “yes” to fully unroll the loop. A loop must be bounded to fully unroll it.

Care must be taken when coding the loop body of an unrolled loop since it is easy to create dependent logic, resulting in longer latency, larger area, and longer HLS runtimes.

Avoid the following:

- Unrolled loops with conditional break
- Unrolled loops with conditional assignments
- Unrolled loops with saturating accumulators

The scenarios describe above can usually be avoided by rewriting the loop body code to be unconditional.

The following example shows a loop that is fully unrolled to accumulate the values of an array. The loop has a conditional break which results in a dependency chain of adders when the loop is unrolled. This means longer latency and larger area.

```
// Unrolled Loop w/ Conditional Break
ac_int<13,false> acc = 0;
ac_int<10,false> data[16];
...
#pragma unroll yes
for(int i=0; i<16; i++){
    acc += data[i];
    if(i == size -1)
        break;
}
```

The following example shows how the previous example can be rewritten to make the loop body unconditional. This coding style should always be used, if possible, when you know that the loop is to be unrolled. The loop exit condition “if(i == size-1)” and break are removed and the “?” operator is used to always accumulate, either “data[i]” or “0” if the conditional is false.

Making the loop body, or assignments in the loop body, unconditional will allow HLS to balance and optimize expressions. In this example it will result in a balanced adder tree.

```
// Unconditional Loop Body
ac_int<13,false> acc = 0;
ac_int<10,false> data[16];
...
#pragma unroll yes
for(int i=0; i<16; i++){
    acc += (i <= size -1) ? data[i] : ac_int<10,false>(0);
}
```

NOTE: Catapult HLS users should always run Design Analyzer on the design and look at the Advisor view. This will identify many of these coding style issues!

2.4.2. Unrolling a Loop to Make the Loop Iterator Constant

It was shown earlier that HLS is able to optimize constants, leading to smaller area. This is especially true when dynamically indexing arrays mapped to registers inside of an unrolled loop.

Dynamically indexing writes to register arrays inside of unrolled loops is far more expensive than dynamically indexing reads. This is because unrolling the loop is creating multiple copies of the dynamically indexed writes to the register array. So, if possible, code should always be transformed so that the write indices are constant. Dynamic indexing of register array reads is much cheaper in terms of area.

These types of transformations can be done by taking advantage of the effects of loop unrolling on the loop iterator.

NOTE: A fully unrolled loop will result in the loop iterator becoming constant!

The following example shows how one array is copied to another using a variable “offset” to copy values to higher address locations than the input array. The variable “offset” is dynamic (or programmable) which results in multiple copies of the dynamic writes created due to the loop being fully unrolled. This will lead to larger area and longer runtimes when synthesized in HLS.

```
uint8 data[20];
uint8 data_off[20];
uint5 offset = offset_in;
#pragma unroll yes
for(int i=0;i<20;i++){
    data[i] = data_in[i];
    data_off[i] = 0;
}
#pragma unroll yes
for(int i=0;i<20;i++){
    if(i+offset<20)
        data_off[i + offset] = data[i];
}
```

The following example shows how the previous example can be rewritten to make the write indices to “data_off” constant. An additional loop has been added that is fully unrolled and compares the value of the

loop iterator “j” to “j == offset”. The array “data_off” is written using the iterators “i+j” when the condition “j==offset && j+i<20”. The index “j+i” becomes constant when both loops are fully unrolled, resulting in constant indices used for the register array writes. This results in a very efficient hardware structure.

```
uint8 data[20];
uint8 data_off[20];
uint5 offset = offset_in;
#pragma unroll yes
for(int i=0;i<20;i++){
    data[i] = data_in[i];
    data_off[i] = 0;
}

#pragma unroll yes
for(int i=0;i<20;i++){
    #pragma unroll yes
    for(int j=0;j<20;j++){
        if(j==offset && j+i < 20)
            data_off[i + j] = data[i];
    }
}
```

2.4.3. Achieving Maximum Throughput

Loop pipelining is used to tune the throughput of a design. Most HLS scheduling failures are due to the loop pipelining constraints being impossible to achieve. These failures are typically related to the use of bandwidth limiting resource inside of a loop such as MatchLib IO Push/Pop and/or memory reads/writes. Please refer to the User Docs for an in-depth discussion of loop pipelining.

Loop Pipelining pragma

Loop pipelining constraints can be applied as a tool directive or as a pragma in the source code.

The loop pipelining pragma takes the form:

```
#pragma hls_pipeline_init_interval <1,2,3,etc, where 1 is max pipelining>
```

It was shown previously that loops could be used to control data movement by using MatchLib Connections to stream data into an out of a design. This coding style combined with loop pipelining can allow designers to get the desired throughput.

The following example shows how the “for” loop that reads and writes MatchLib Connections interfaces is pipelined with II=1. This means that the synthesized design will be able to read and write once per clock cycle. It should be noted that the pipelining pragma could also have been applied to the function call itself for the maximum performance. In this example the “for” loop will ramp-up and ramp-down as it is entered and exited.

```
#pragma hls_pipeline_init_interval 1
```

```
for(int i=0; i<10; i++){
    ac_int<10,false> data = din.Pop();
    dout.Push(data);
}
```

Limit reads/writes to MatchLib Connections I/F or Arrays mapped to Memories

The minimum achievable pipeline initiation interval is dependent on the number of bandwidth-limiting resources inside a loop body. A pipelining constraint that cannot be satisfied will result in a scheduling failure during HLS synthesis.

The following example design that accumulates every two values read from the MatchLib IO “din” and has a pipeline constraint with the initiation interval equal to 1 (II=1). This design will fail to schedule because there are 2 reads to the MatchLib interface “din” inside of the loop body. At best it could be pipelined with II=2.

```
#pragma hls_pipeline_init_interval 1
for(ac_int<10,false> i=0; i<1024; i+=2){
    ac_int<10,false> acc = 0;
    acc = din.Pop();
    acc += din.Pop();
    dout.Push(acc);
}
```

The following example shows how the previous design can be rewritten to allow pipelining with II=1. The reading of “din” has been reduced to one read per loop iteration. Additional code has been added to ensure that every two reads are accumulated and written out.

```
// Coding for II=1
ac_int<10,false> acc;//data member is persistent, initialized to 0
SC_CTOR(mac){
}
...
#pragma hls_pipeline_init_interval 1
for(ac_int<10,false> i=0; i<1024; i+=1){
    acc += din.Pop();//read once per loop iteration
    if(i&1){//write and clear on even loop iterations
        dout.Push(acc);
        acc = 0;
    }
}
```

2.4.4. Inter-Process Concurrency

There are primarily two reasons for partitioning a design into multiple concurrent processes; partitioning based on functionality to reduce complexity, and partitioning to increase performance by allowing different parts of the design/algorithm run concurrently in hardware (This is often the case when a design has non-mergable sequential loops that are left “rolled”). These two cases result in two or more interconnected

processes where the interconnect is either signals (for signal-level design), MatchLib Connections combinational channels, or shared memory interconnect with inter-process synchronization.

Multiple Interconnected Processes with Connections Channel

The following design example shows how the MatchLib Connections Combinational channel can be used to interconnect two processes.

```

1
2 #include <ac_sysc_macros.h>
3 #include <mc_connections.h>
4 #pragma hls_design top
5 class mult_add : public sc_module
6
7     class two_process : public sc_module
8     {
9     public:
10         sc_in<bool> CCS_INIT_S1(clk);
11         sc_in<bool> CCS_INIT_S1(rstn);
12
13         Connections::In <ac_int<16,false>> CCS_INIT_S1(din);
14         Connections::Out<ac_int<16,false>> CCS_INIT_S1(dout);
15         Connections::Combinational<ac_int<16,false>> CCS_INIT_S1(connect);
16         SC_CTOR(two_process) {
17             SC_THREAD(writer);
18             sensitive << clk.pos();
19             async_reset_signal_is(rstn, false);
20             SC_THREAD(reader);
21             sensitive << clk.pos();
22             async_reset_signal_is(rstn, false);
23         }
24     private:
25         void writer() {
26             din.Reset();
27             connect.ResetWrite();
28             wait();
29             #pragma hls_pipeline_init_interval 1
30 #pragma pipeline_stall_mode flush
31             while (1) {
32                 ac_int<16,false> data = din.Pop();
33                 connect.Push(data + 0x100);
34             }
35         }
36         void reader() {
37             dout.Reset();
38             connect.ResetRead();
39             wait();
40             #pragma hls_pipeline_init_interval 1
41 #pragma pipeline_stall_mode flush
42             while (1) {
43                 ac_int<16,false> data = connect.Pop();
44                 dout.Push(data - 0x100);
45             }
46         }
47     };
PATH: $MGC_HOME/shared/examples/docs/qstart/sysc/area/two_process.h

```

The following list highlights the key areas of the previous design:

- Line 8 defines a MatchLib Combinational channel to connect the two processes
- Lines 11 through 15 define two processes, writer and reader
- Line 20 of the writer process resets the write side on the channel
- Line 26 of the write process writes to the channel
- Line 31 resets the read side of the channel
- Line 37 of the read process reads from the channel

Sequential Loop Behavior

One of the most common reasons for partitioning a design into multiple processes is when the design contains multiple dependent sequential loops. Fully dependent (second loop depends on data from the first loop after it completes) sequential loops will execute sequentially in hardware, leading to lower performance.

The following example shows a pipelined design that contains sequential loops. The first loop writes an array mapped to a single-port memory in ascending order. The second loop reads the register array in descending order. This design will take ~128 clock cycles to execute in hardware because the second loop must wait for the first loop to finish executing. These loops should be pushed into separate threads so that they can be synthesized as concurrent processes to improve performance. Doing this requires coding the design to interconnect the separate processes using a shared SystemC memory along with some synchronization logic.

```

1
2 #include <ac_sysc_macros.h>
3 #include <mc_connections.h>
4 #pragma hls_design top
5 class reverse : public sc_module
6 {
7 public:
8     sc_in<bool> CCS_INIT_S1(clk);
9     sc_in<bool> CCS_INIT_S1(rstn);
10    Connections::In <ac_int<16,false>> CCS_INIT_S1(din);
11    Connections::Out<ac_int<16,false>> CCS_INIT_S1(dout);
12    SC_CTOR(reverse) {
13        SC_THREAD(main);
14        sensitive << clk.pos();
15        async_reset_signal_is(rstn, false);
16    }
17 private:
18     void main() {
19         din.Reset();
20         dout.Reset();
21         ac_int<16> mem[64]; //mapped to sp ram
22         wait();
23         #pragma hls_pipeline_init_interval 1
24 #pragma pipeline_stall_mode flush
25         while (1) {
26             for (int i=0; i<64; i++) {
27                 mem[i] = din.Pop();
28             }
29             for (int i=63; i>=0; i--) {

```

```

30     dout.Push(mem[i]);
31     }
32     }
33 }
34 };
PATH: $MGC_HOME/shared/examples/docs/qstart/sysc/area/loops_seq.h

```

2.4.5. Coding Memory Architecture

High-Level Synthesis synthesizes the architecture described in the C++ design. Multiple micro-architectures can be generated from this C++ description using different synthesis constraints, but HLS cannot add an underlying memory architecture that is not described in the source.

One of the most common reasons for sub-optimal performance is due to a lack of memory architecture described in the C++. Poorly described, or not described at all, memory architectures tend to limit loop pipelining and unrolling optimizations.

A very simple example of this is a moving average filter that averages 3 sequential data values from an array.

```

1
2 #include <ac_fixed.h>
3 #include "types.h"
4 class moving_avg_1d_algo
5 {
6     int clip(int idx) { //Boundary processing
7         if (idx < 0)
8             { return 0; }
9         else if (idx > 99)
10            { return 99; }
11        else
12            { return idx; }
13    }
14 public:
15     moving_avg_1d_algo() {}
16
17     #pragma hls_design interface
18     void run(ac_fixed<8,8,false> din[100], ac_fixed<8,8,false> dout[100]) {
19         ac_fixed<12,8,false> acc = 0;
20         AVG:for (int i=0; i<100; i++) {
21             acc = din[clip(i-1)]*k[0] + din[i]*k[1] + din[clip(i+1)]*k[2];
22             dout[i] = acc;
23         }
24     }
25 };
PATH: $MGC_HOME/shared/examples/docs/qstart/sysc/mem/moving_avg_1d.cpp

```

The problem with the previous implementation is that mapping the “din” array to a single port memory will prevent pipelining the design with II=1. This is because pipelining with II=1 implied starting a new iteration of the AVG loop every clock cycle. This means reading 3 values from “din” in parallel every clock cycle, which is impossible if “din” has a single memory read port.

The key to designing efficient memory architectures that can be optimized efficiently by HLS is to code the C++ memory architecture so that the internal bandwidth of the C++ matches the bandwidth of limiting resources such as memories, memory interfaces, or IO. In this example the code needs to be rewritten to reduce the reading of “din” to once per loop iteration of the AVG loop.

Analysis of the code shows that each loop iteration reuses two of the values read in the previous iteration. Storing these two values (shift register storage) locally allows the design to be rewritten so that “din” is only read once per loop iteration. This type of architecture is known as a “Sliding Window” architecture. This is shown in the following example:

```

1
2 #include <ac_fixed.h>
3 class moving_avg_1d_arch
4 {
5 public:
6     moving_avg_1d_arch() {}
7
8     #pragma hls_design interface
9     void run(ac_fixed<8,8,false> din[100], ac_fixed<8,8,false> dout[100]) {
10         ac_fixed<12,8,false> acc = 0;
11         ac_fixed<8,8,false> dat0,dat1,dat2;
12         ac_fixed<8,8,false> win0,win1,win2;
13
14         AVG:for (int i=0; i<101; i++) {
15             if (i != 100)
16                 { dat0 = din[i]; }
17             //boundary
18             if (i==1)
19                 { win2 = dat1; }
20             else
21                 { win2 = dat2; }
22             if (i==100)
23                 { win0 = dat1; }
24             else
25                 { win0 = dat0; }
26             win1 = dat1;
27
28             acc = win2*k[0] + win1*k[1] + win0*k[2];
29             //Shift register
30             dat2 = dat1;
31             dat1 = dat0;
32             if (i != 0)
33                 { dout[i-1] = acc; }
34         }
35     }
36 };
PATH: $MGC_HOME/shared/examples/docs/qstart/sysc/mem/moving_avg_1d_arch.cpp

```

The moving average example of Architecture showed that a shift-register architecture could be used to store a “window” of data for a 1-d array input to achieve higher performance by being able to pipeline with II=1. This same approach can be used for the slightly more complicated example when the input is a 2-d array and the moving average filter averages data across different rows on the input image. This algorithm is shown below:

```

1
2 #include <ac_fixed.h>
3 class moving_average_2d_algorithm
4 {
5     int clip(int idx) {
6         if (idx < 0)
7             { return 0; }

```

```

8     else if (idx > 99)
9     { return 99; }
10    else
11    { return idx; }
12  }
13 public:
14  moving_average_2d_algorithm() {}
15
16  #pragma hls_design interface
17  void run(ac_fixed<8,8,false> din[100][100],
18          ac_fixed<8,8,false> dout[100][100]) {
19      ac_fixed<12,8,false> acc = 0;
20      ROW:for (int i=0; i<100; i++) {
21          COL:for (int j=0; j<100; j++) {
22              acc = din[clip(i-1)][j]*k[0] + din[i][j]*k[1] + din[clip(i+1)][j]*k[2];
23              dout[i][j] = acc;
24          }
25      }
26  }
27 };
PATH: $MGC_HOME/shared/examples/docs/qstart/sysc/mem/moving_avg_2d.cpp

```

The 2-D moving average algorithm will have the same type of performance bottleneck as the 1-D example when “din” is mapped to a single port memory. Pipelining with $II=1$ will not be possible because there are 3 reads of “din” for each loop iteration of the COL loop. The main difference between the 2-D and 1-D moving average algorithms is that the 2-D case is reading from 3 different rows of data for each iteration of the “COL” loop. This means that a shift-register sliding window architecture is not the solution. The desired architecture needs to buffer multiple rows of data in order to be able to achieve the best-case performance. This means coding a line-buffer sliding window memory architecture where separate internal memories are used to store row data from the input array. This is essentially a line-buffer sliding window memory architecture and is shown in the following example. This code rewrites adds two arrays `line_buf0` and `line_buf1`, and the row data from “din” is shifted through the arrays. The C++ is structured so that the arrays are only read or written once per COL loop iteration, which allows them to be mapped to a memory with one read port and one write port.

```

1
2 #include <ac_fixed.h>
3 class moving_average_2d_architecture
4 {
5 public:
6     moving_average_2d_architecture() {}
7
8     #pragma hls_design interface
9     void run(ac_fixed<8,8,false> din[100][100], ac_fixed<8,8,false> dout[100][100]) {
10         ac_fixed<12,8,false> acc = 0;
11         ac_fixed<8,8,false> dat0,dat1,dat2;
12         ac_fixed<8,8,false> win0,win1,win2;
13         ac_fixed<8,8,false> line_buf0[100]; //mapped to memory
14         ac_fixed<8,8,false> line_buf1[100]; //mapped to memory
15
16         ROW:for (int i=0; i<101; i++) {
17             COL:for (int j=0; j<100; j++) {
18                 if (i != 100)
19                 { dat0 = din[i][j]; }
20                 dat2 = line_buf1[j];
21                 dat1 = line_buf0[j];
22                 //boundary

```

```

23     if (i==1)
24     { win2 = dat1; }
25     else
26     { win2 = dat2; }
27     if (i==100)
28     { win0 = dat1; }
29     else
30     { win0 = dat0; }
31     win1 = dat1;
32
33     acc = win2*k[0] + win1*k[1] + win0*k[2];
34     //Shift memories
35     line_buf1[j] = line_buf0[j];
36     line_buf0[j] = dat0;
37     if (i != 0)
38     { dout[i-1][j] = acc; }
39 }
40 }
41 }
42 };
PATH: $MGC_HOME/shared/examples/docs/qstart/sysc/mem/moving_avg_2d_arch.cpp

```

2.5. Non-Blocking IO

2.6. Coding for Power

Just like coding for area, designers need to be explicit about the architecture described in the C++ when power consumption is a consideration. This means structuring the code so that switching is minimized when possible. When coding for power users need to think about how the design is going to be optimized in Catapult HLS. Are loops being unrolled or left rolled? Are arrays being mapped to memories or registers? Understanding how the design will be synthesized, and what the expected hardware structure is, should help determine how to structure the C++ code. Catapult is able to analyze and optimize for power and coding style will have a large impact on the results.

A simple set of examples that illustrates this are discussed below.

The first example is a parallel multiply-accumulate that is fed by a parallel shift register “regs”, where the “regs” array is mapped to registers. In this example all loops are unrolled and the number of multiply-accumulates is programmable based on an interface variable “size”. This design is not efficient for power consumption as it always is shifting all 31 elements of the register “regs” regardless of the value of “size”, which determines the number of multiply-accumulates.

```

1
2
3 void parallel_mac()
4 {
5     int regs[31];
6     wait();
7     while (1) {
8         int acc = 0;
9 #pragma unroll yes

```

```

10     for (int i=30; i>=0; i--) {
11         if (i==0)
12             { regs[i] = din.Pop(); }
13         else
14             { regs[i] = regs[i-1]; }
15     }
16 #pragma unroll yes
17     for (int i=0; i<31; i++) {
18         acc += (i<size) ? regs[i]*coeffs[i] : 0;
19     }
20     dout.Push(acc);
21 }
22 }
PATH: $MGC_HOME/shared/examples/docs/qstart/sysc/power/parallel_mac.cpp

```

A more power efficient way to code this functionality is to gate the shifting of the data based on the “size” configuration input. This is shown in the following example:

```

1
2
3 void parallel_mac_opt()
4 {
5     int regs[31];
6     wait();
7     while (1) {
8         int acc = 0;
9 #pragma unroll yes
10        for (int i=30; i>=0; i--) {
11            if (i==0)
12                { regs[i] = din.Pop(); }
13            else if (i<size-1)
14                { regs[i] = regs[i-1]; }
15        }
16 #pragma unroll yes
17        for (int i=0; i<31; i++) {
18            acc += (i < size) ? regs[i]*coeffs[i] : 0;
19        }
20        dout.Push(acc);
21    }
22 }
PATH: $MGC_HOME/shared/examples/docs/qstart/sysc/power/parallel_mac_opt.cpp

```

The two previous examples showed how switching can be minimized by gating off the shift register based on the configuration input “size”. However, in these examples the multiply-accumulate loop was fully parallelized. Let’s look at the case where the multiply-accumulate loop is left rolled.

The un-optimized version is shown below:

```

1
2 #include <systemc.h>
3 void seq_mac()
4 {
5     int regs[31];
6     wait();
7     while (1) {
8         int acc = 0;

```

```

9 #pragma unroll yes
10 for (int i=30; i>=0; i--) {
11     if (i==0)
12         { regs[i] = din.Pop(); }
13     else
14         { regs[i] = regs[i-1]; }
15 }
16 //Loop is left rolled
17 for (int i=0; i<31; i++) {
18     acc += regs[i]*coeffs[i];
19     if (i==size-1)
20         { break; }
21 }
22 dout.Push(acc);
23 }
PATH: $MGC_HOME/shared/examples/docs/qstart/sysc/power/seq_mac.cpp

```

Compared to the previous example, the multiply-accumulate loop in the next example has been written with a conditional break based on “size” that allows the loop to exit early. This is more efficient since the loop is left “rolled”. However, this implementation is not power efficient since the shift register “regs” shifts all elements for each new data value read from “din”. A better implementation is to use a circular buffer implementation for the shift register so that only one element of “regs” is ever written for each new value of “din”, reducing the switching.

```

1
2 #include <systemc.h>
3 void seq_mac_opt()
4 {
5     int regs[31];
6     uint5 idx_wr=0, idx_rd=0;
7     wait();
8     while (1) {
9         int acc = 0;
10
11         //Circular Buffer
12         regs[idx_wr] = din.Pop();//one write to regs[idx_wr]
13         idx_wr++;
14         if (idx_wr == 31)
15             { idx_wr = 0; }
16
17         //Loop is left rolled
18         for (int i=0; i<31; i++) {
19             idx_rd = idx_wr;
20             acc += regs[idx_rd]*coeffs[i];
21             if (idx_rd == 0)
22                 { idx_rd = 30; }
23             else
24                 { idx_rd--; }
25             if (i==size-1)
26                 { break; }
27         }
28         dout.Push(acc);
29     }
30 }
PATH: $MGC_HOME/shared/examples/docs/qstart/sysc/power/seq_mac_opt.cpp

```

This same circular buffer approach can also be used when designing more complicated memory

architectures that require memories to buffer intermediate data.

2.7. Using HLS Math Libraries

High-level synthesis is used to turn algorithms written in C++ into high-quality RTL. One of the “sweet spots” for HLS is for compute intensive algorithms in communications, image and video processing, and machine learning, to name a few. Many of the algorithms in these domains require math functions such as square root, arc tangent, sine, cosine, etc. These functions are available in the standard C++ `math.h` library. However, Catapult HLS does not directly synthesize `math.h`. Instead, it provides highly optimized synthesizable versions of the most commonly used functions in `math.h`. These synthesizable versions are implemented using piecewise linear, CORDIC and recursive algorithms for a range of area/performance tradeoffs. These libraries are available both in the Catapult install tree as well as from hlslibs.org and support math functions for bit-accurate integer, fixed-point and floating-point types.

To use the catapult math libraires, you will need the following include paths

- Compiler include path for Catapult's include directory

```
<Catapult Install Tree>/Mgc_home/shared/include
```

- Includes added to the C++ source

```
#include <math/mgc_ac_math.h>
#include <ac_math.h>
```

The following example demonstrates using the synthesizable `math.h` library for integer square root.

```
1
2 #include <ac_math.h>
3 #include <math/mgc_ac_math.h>
4 #pragma hls_design top
5 void test_math(ac_int<10,false> din, ac_int<6,false> &dout)
6 {
7     ac_math::ac_sqrt(din,dout);
8 }
PATH: $MGC_HOME/shared/examples/docs/qstart/sysc/math/test_math.h
```

Chapter 3: Third-Party Information

The following HTML page provides information on third-party software that may be included in the Catapult product, including any additional license terms. This document is located at the following location within the Catapult install tree:

```
$MGC_HOME/shared/pkgs/sif_docs/legal/Catapult_OSS.html
```